

BIBLIA

Uso moderno de inteligencia artificial, subagentes, SDD, AGENTS.md, GitHub, prompt engineering y harness engineering.

Autoría

Nicolás Ezequiel Melluso

nicolas.e.melluso@gmail.com

linkedin.com/in/nicolas-ezequiel-melluso

github.com/Nicolas-Melluso

Índice general

01	Uso Moderno de Inteligencia Artificial	3
	Cómo pasar de chatear con un modelo a trabajar con un sistema de pensamiento, ejecución y verificación	
02	Uso Inteligente de Subagentes	14
	Criterios, patrones y cierre operativo para delegar mejor	
03	SDD y Estructura de Soporte	27
	Specification-Driven Development aplicado a repositorios con IA	
04	AGENTS.md, .github y Comandos Estilo Prompt	43
	Cómo construir un repositorio preparado para agentes, SDD, Copilot, workflows y prompts reutilizables	
05	Prompt Engineering y Harness Engineering	60
	De prompts sueltos a sistemas versionados, evaluables y productivos	
06	Ejemplos y Casos de Uso	79
	Scaffolding completo para aplicar La Biblia Moderna en un repositorio real	

RUTA PRÁCTICA

Si ya tenés un repo en marcha, empezá por los volúmenes 03, 04 y 05. Si estás armando criterio desde cero, leé primero 01 y 02. Para copiar una base completa, cerrá con el volumen 06.

VOLUMEN 01

Uso Moderno de Inteligencia Artificial

Cómo pasar de chatear con un modelo a trabajar con un sistema de pensamiento, ejecución y verificación

Al terminar este volumen podés distinguir entre usar IA como chat y usarla como sistema de trabajo.

La idea central

Usar inteligencia artificial de forma moderna no es abrir un chat, pedir "haceme esto" y aceptar la primera respuesta. Eso fue la primera etapa. La etapa moderna es tratar a la IA como una capa de trabajo: una combinación de asistente, par técnico, investigador, ejecutor, revisor y sistema de memoria. La diferencia no está en escribir prompts más largos. Está en diseñar un flujo donde cada interacción deja contexto, artefactos, pruebas y decisiones reutilizables.

La forma inmadura de usar IA es conversacional y descartable: se pregunta algo, se obtiene una respuesta, se copia una parte y se sigue. La forma madura es operacional: se define el objetivo, se entrega contexto, se explicitan restricciones, se divide el trabajo, se verifica el resultado y se guarda lo aprendido. El valor aparece cuando la IA deja de ser una máquina de texto y empieza a funcionar como una extensión del proceso de desarrollo, investigación o producción.

Este tomo propone un modelo de trabajo simple: pensar en la IA como un sistema con cuatro capas.

1. Capa de contexto: lo que la IA debe saber antes de actuar.
2. Capa de tarea: lo que debe producir ahora.
3. Capa de verificación: como se sabe que el resultado sirve.
4. Capa de memoria: donde queda registrado para no repetir el mismo razonamiento.

Cuando esas cuatro capas existen, la IA puede ayudar en trabajos serios: escribir especificaciones, revisar código, comparar alternativas, generar documentación, ejecutar pruebas, detectar riesgos, preparar presentaciones, entrenar a una persona o acelerar decisiones técnicas. Cuando faltan, la IA se vuelve brillante por momentos y peligrosa en silencio.

Lo que cambio en la práctica

El cambio importante no es solo que los modelos sean mejores. El cambio es que ahora se puede trabajar con agentes conectados a herramientas: editor, terminal, navegador, repositorio, issue tracker, documentación, bases de datos locales, tests, linters, emuladores y pipelines. Antes el modelo respondía desde afuera del trabajo. Ahora puede participar dentro del trabajo.

Eso obliga a cambiar la forma de pedir. Un pedido moderno no dice solo "explicame X". Dice: "lee estos archivos, identifica el comportamiento actual, propone un cambio acotado, implementalo, corre estas pruebas y dejame un resumen con riesgos". La IA deja de ser un oráculo y pasa a ser un operador bajo contrato.

También cambia el rol humano. El humano ya no gana por tipear todo manualmente. Gana por definir buen contexto, buenas restricciones, buenos criterios de aceptación y buenos mecanismos de verificación. La IA puede producir mucho, pero no sabe sola que tradeoff conviene para tu negocio, que riesgo legal aceptar, que deuda técnica tolerar o que experiencia de usuario querés defender.

La pregunta moderna no es "que prompt uso". La pregunta moderna es "que sistema de trabajo hace que los resultados sean mejores cada semana".

El ciclo de trabajo recomendado

Un flujo robusto con IA puede verse así:

```
Intención -> Contexto -> Plan -> Ejecución -> Verificación -> Registro -> Siguiendo iteración
```

La intención define el resultado deseado. El contexto reduce ambigüedad. El plan evita trabajo impulsivo. La ejecución produce artefactos concretos. La verificación separa lo convincente de lo correcto. El registro evita que el conocimiento se pierda en el chat. La siguiente iteración convierte el trabajo en aprendizaje acumulado.

Un ejemplo simple:

Intención:

Quiero agregar autenticación por magic link.

Contexto:

Repo Node/TypeScript, PostgreSQL, arquitectura por servicios, tests con Vitest.

Plan:

1. Ubicar auth actual.
2. Agregar tabla de tokens.
3. Implementar servicio.
4. Agregar pruebas unitarias e integración.
5. Documentar variables de entorno.

Verificación:

```
npm test
npm run typecheck
prueba manual del flujo login -> link -> sesión
```

Registro:

ADR corta sobre por qué magic link y no password.
Spec de comportamiento esperado.
Checklist de rollback.

Ese flujo no depende de una herramienta específica. Sirve con Codex, Copilot, Cursor, Claude Code, Gemini CLI o un agente propio. La madurez está en el proceso.

La unidad mínima de contexto

La IA trabaja mejor cuando recibe contexto empaquetado, no una nube de información. La unidad mínima de contexto para una tarea sería debería incluir:

Elemento	Para que sirve	Ejemplo
Objetivo	Evita que optimice otra cosa	"Reducir errores de checkout abandonado"
Estado actual	Le da punto de partida	"El retorno de Stripe vuelve a <code>/checkout/success</code> "
Restricciones	Acota soluciones	"No tocar precios ni migrar proveedor"
Archivos relevantes	Reduce exploracion ciega	<code>src/server.js</code> , <code>public/app.js</code>
Criterios de aceptación	Define cierre	"Si vuelve de pago, muestra estado comprado"
Verificación	Obliga a probar	"Smoke test local y test unitario"
Riesgos	Hace visible lo delicado	"No filtrar secretos en logs"

Sin esa unidad mínima, el modelo completa huecos con supuestos. A veces acierta. En sistemas reales, a veces rompe cosas por seguir una lógica que parecía razonable pero no pertenecía al proyecto.

Buenas tareas para IA

La IA es especialmente fuerte cuando puede operar sobre información explícita y cuando el resultado puede verificarse. Algunos usos de alto valor:

1. Resumir y mapear código existente.
2. Convertir una idea en una especificación revisable.
3. Generar casos de prueba desde criterios de aceptación.
4. Detectar inconsistencias entre docs, código y tests.
5. Refactorizar una zona acotada con suite de pruebas.
6. Preparar scripts de migración o validación.
7. Crear runbooks para operaciones repetibles.
8. Revisar PRs con reglas concretas.
9. Convertir conversaciones en tareas accionables.
10. Crear material de entrenamiento para otra persona.

La IA es menos confiable cuando se le pide decidir sin datos, inventar políticas de negocio, tocar muchas zonas del sistema a la vez, cambiar infraestructura sin permisos o producir contenido "definitivo" sin revisión humana. No significa que no pueda ayudar. Significa que hay que darle un marco de control más fuerte.

El prompt moderno

Un prompt moderno tiene forma de brief operativo. No necesita ser poético ni enorme. Necesita eliminar ambigüedad.

Objetivo:

Quiero que conviertas esta idea en una especificación SDD lista para implementar.

Contexto:

El producto es una app B2B para gestionar reclamos. El repo usa Node.js, TypeScript, PostgreSQL y GitHub Actions. Queremos mantener el alcance chico.

Entrada:

Idea: "permitir que un operador reasigne un reclamo a otro equipo".

Restricciones:

- No diseñar una pantalla completa todavía.
- No asumir roles nuevos si no son necesarios.
- Separar reglas de negocio de UI.
- Incluir riesgos y preguntas abiertas.

Salida:

1. Resumen del problema.
2. Requisitos funcionales.
3. Requisitos no funcionales.
4. Criterios de aceptación.
5. Casos borde.
6. Plan de implementación en 3 slices.
7. Pruebas recomendadas.

Calidad:

Si falta información, marca como pregunta abierta. No inventes datos de negocio.

Ese formato hace tres cosas: le da dirección, le da límites y define cómo evaluar la respuesta. No busca "inspirar" al modelo. Busca contratarlo para una tarea.

De chat a repositorio

El salto más importante es mover conocimiento desde el chat hacia el repositorio. El chat es frágil: se pierde, se contradice, no versiona bien, no se revisa en PR y no corre en CI. El repositorio, en cambio, puede guardar instrucciones, specs, decisiones, prompts, tests y workflows.

Una organización moderna suele separar:

Artefacto	Audiencia	Función
<code>README.md</code>	Humanos nuevos	Presentar el proyecto y como empezar
<code>AGENTS.md</code>	Agentes de código	Reglas operativas, comandos, estilo y verificación
<code>.github/copilot-instructions.md</code>	Copilot	Instrucciones generales para respuestas en el repo
<code>.github/instructions/*.instructions.md</code>	Copilot por path	Reglas específicas por zona del código
<code>.github/prompts/*.prompt.md</code>	Humanos y asistentes	Comandos reutilizables estilo prompt
<code>.github/orquestador/sdd/*</code>	Equipo y agentes	Specs, decisiones, tareas, trazabilidad
<code>.github/workflows/*</code>	CI/CD	Automatización ejecutable

La regla práctica: lo que se repite debe vivir como archivo. Si cada vez que pedis ayuda tenés que explicar los mismos comandos, el mismo estilo, las mismas restricciones y los mismos criterios de prueba, eso no es prompt engineering. Es deuda de contexto.

El repositorio como sistema operativo del agente

Una regla fuerte ordena todo el enfoque: un agente no debería trabajar desde una conversacion; debería trabajar dentro de un **harness**. El chat coordina, pero el repositorio conserva la verdad operativa.

Un harness es el conjunto de archivos, estados, specs, scripts, permisos, subagentes, tests y revisiones que hace que la IA pueda trabajar sin depender de memoria oral. La diferencia es concreta:

```
Chat = coordinacion
Repo = verdad
Specs = contrato
Tests = evidencia
CI/hooks = verificación
Progress = memoria de ejecución
```

Esto cambia el rol del repositorio. Ya no es solo el lugar donde vive el código. También es el lugar donde vive el proceso. Una feature sería debería poder responder, desde archivos:

1. Qué se está construyendo.
2. Qué spec fue aprobada.
3. Qué tareas quedan pendientes.
4. Qué tests cubren cada requirement.
5. Qué hizo el implementador.
6. Qué rechazó o aprobó el reviewer.

7. Cómo se valida el estado final.

Los buenos prompts, el buen contexto y la buena verificación no alcanzan si el proceso puede saltarse sin dejar rastros. Si el trabajo importa, el agente no debería poder saltarse el proceso sin dejar evidencia.

Una estructura mínima de harness puede verse así:

```
repo/  
  AGENTS.md  
  feature_list.json  
  init.sh  
  CHECKPOINTS.md  
  docs/  
    architecture.md  
    conventions.md  
    specs.md  
    verification.md  
  specs/  
    <feature>/  
      requirements.md  
      design.md  
      tasks.md  
  progress/  
    current.md  
    history.md  
    impl_<feature>.md  
    review_<feature>.md
```

No todos los proyectos necesitan exactamente esa forma, pero sí necesitan esas responsabilidades. Si el estado importante está solo en el chat, el sistema todavía es frágil.

Ejemplo de flujo completo:

1. El issue describe "agregar comando recent".
2. `requirements.md` define R1 y R2 en lenguaje verificable.
3. `design.md` decide tocar `src/cli.py` y `tests/test\_cli.py`.
4. `tasks.md` parte el trabajo en T1, T2 y T3.
5. El implementador cambia código y tests.
6. El reviewer valida R1 -> test_recent_default_limit y R2 -> test_recent_invalid_limit.
7. `init.sh` corre verde.
8. `progress/history.md` registra cierre y evidencia.

Lo importante no es la carpeta exacta. Lo importante es que la idea no salte directo de una conversacion a código. Pasa por contrato, tarea, prueba, revisión y evidencia.

Roles de IA en un equipo chico

Aunque una sola herramienta parezca "un asistente", conviene pensar en roles:

Rol	Qué hace	Buen output
Investigador	Lee, compara, resume, encuentra patrones	Mapa de archivos, riesgos, preguntas
Planner	Divide el trabajo	Plan por slices, dependencias, verificación
Implementador	Cambia archivos	Patch acotado, tests, resumen
Revisor	Busca errores	Findings con archivo y línea
Documentador	Convierte trabajo en conocimiento	README, ADR, runbook
Evaluador	Prueba comportamiento	Reporte de comandos y resultados

Los subagentes formalizan esa separación, pero el modelo mental sirve incluso con un solo chat. Cuando una misma conversación intenta hacer todo al mismo tiempo, se vuelve confusa. Cuando cada rol tiene una salida concreta, el trabajo se vuelve controlable.

La verificación no es opcional

La IA puede producir texto convincente y código que parece correcto. La única defensa seria es verificar. La verificación puede ser automática o humana, pero debe existir.

Ejemplos de verificación:

1. Tests unitarios y de integracion.
2. Typecheck, lint y build.
3. Smoke test manual documentado.
4. Comparacion contra criterios de aceptación.
5. Revisión de diffs archivo por archivo.
6. Prueba con datos reales o fixtures.
7. Evaluación con golden cases para prompts.
8. Checklist de seguridad y permisos.

La frase "se ve bien" no alcanza. Un flujo moderno pide que la IA diga que ejecuto, que no pudo ejecutar, que cambio, que falta y que riesgo queda.

Memoria útil, no memoria infinita

La memoria sirve cuando reduce repetición y mejora consistencia. No sirve cuando se convierte en un basural de notas largas. Una memoria útil tiene tres propiedades:

1. Es recuperable: está en una ruta conocida.
2. Es accionable: contiene decisiones, comandos, convenciones o errores aprendidos.

3. Es verificable: no reemplaza al estado actual del repo cuando este puede haber cambiado.

Ejemplos de memoria buena:

- El repo usa ``.github/orquestador`` como carpeta de contexto.
- Los workflows son la única capa ejecutable; el catálogo solo documenta.
- Antes de cerrar cambios de runtime correr ``npm test`` y ``npm run build``.
- En Windows, verificar locks antes de renombrar carpetas.

Ejemplos de memoria mala:

- El proyecto es importante.
- A veces falla.
- Usar buenas practicas.

La memoria moderna no guarda sentimientos. Guarda operaciones.

Plan de adopción en 4 pasos

Paso 1: ordenar el contexto

Crear `AGENTS.md`, documentar comandos reales, listar restricciones y definir donde vive el SDD. El objetivo no es cubrir todo. Es que un agente pueda entrar al repo y no perder media hora adivinando.

Entregables:

1. `AGENTS.md` inicial.
2. `README.md` actualizado.
3. ``.github/orquestador/context/product.md``.
4. ``.github/orquestador/context/architecture.md``.
5. Lista de comandos de verificación.

Paso 2: trabajar por especificaciones

Elegir una feature chica y escribir una spec antes de implementar. Incluir criterios de aceptación, casos borde, no objetivos y pruebas. Después pedir a la IA que implemente solo un slice.

Entregables:

1. Primera spec SDD.
2. Plan por slices.
3. ADR si hay una decisión técnica relevante.
4. Tests asociados.

Paso 3: introducir prompts reutilizables

Crear prompts para tareas repetidas: planificar feature, revisar PR, generar tests, escribir ADR, preparar runbook. Guardarlos en `.github/prompts` o en la carpeta de orquestación elegida.

Entregables:

- 1. `plan-feature.prompt.md` .
- 2. `review-pr.prompt.md` .
- 3. `write-adr.prompt.md` .
- 4. `generate-tests.prompt.md` .

Paso 4: medir calidad

Agregar harnesses o evaluaciones simples. No hace falta montar una plataforma enorme. Empezar con fixtures, casos esperados y un script que compare salidas.

Entregables:

- 1. Carpeta `evals/` .
- 2. Fixtures de entrada.
- 3. Rúbrica de evaluación.
- 4. Script local o workflow de validación.

Anti-patrones comunes

Anti-patrón	Sintoma	Correccion
Prompt gigante para todo	El modelo ignora partes	Dividir en instrucciones persistentes y tareas concretas
Sin verificación	Outputs lindos pero fraguados	Definir comandos y criterios de aceptación
Contexto solo en chat	Se repite todo cada sesión	Mover reglas al repo
Agente con permisos amplios	Riesgo de cambios destructivos	Ownership y permisos minimos
Todo en un solo subagente	Paralelismo falso	Separar exploracion, implementación y revisión
Docs que no corren	Buenas intenciones sin efecto	Conectar docs con workflows y checklists

Checklist para usar IA de forma moderna

Antes de pedir:

1. Tengo claro el resultado final.
2. Puedo nombrar los archivos o dominios relevantes.
3. Se que no quiero que toque.
4. Tengo una forma de verificar.
5. Puedo aceptar un primer slice en vez de todo el sistema.

Durante el trabajo:

1. Pido planes cortos para tareas riesgosas.
2. Divido investigación, edicion y revisión.
3. Mantengo ownership de archivos.
4. Leo los diffs antes de cerrar.
5. Registro decisiones nuevas.

Al cerrar:

1. Se que cambio.
2. Se que pruebas pasaron.
3. Se que pruebas no se corrieron.
4. Se que riesgos quedan.
5. El conocimiento reusable quedo en archivos.

Cierre

La IA moderna no reemplaza el oficio. Lo amplifica cuando el oficio está ordenado. El usuario que más obtiene no es el que sabe el truco secreto del prompt, sino el que sabe convertir trabajo ambiguo en unidades verificables.

La regla final es simple: si el output importa, tratá a la IA como parte de un sistema de producción. Dale contexto, límites, herramientas, pruebas y memoria. Lo demas es solo chat.

VOLUMEN 02

Uso Inteligente de Subagentes

Criterios, patrones y cierre operativo para delegar mejor

Al terminar este volumen podés delegar tareas a subagentes con ownership, evidencia y cierre verificable.

Para qué sirve este volumen

Trabajar con agentes y subagentes modernos no consiste en repartir tareas al azar. La diferencia entre una delegación útil y una pérdida de tiempo suele estar en tres cosas: el tamaño del problema, la calidad del encuadre y el nivel de control sobre el resultado. Cuando se hace bien, el trabajo se acelera, el contexto se usa mejor y el hilo principal queda libre para decisiones que realmente requieren criterio humano o arquitectónico.

Este documento propone una forma práctica de usar subagentes en proyectos técnicos. No intenta vender magia ni automatización total. El objetivo es más modesto y más útil: que puedas dividir trabajo con criterio, pedirle a cada subagente lo justo, conservar trazabilidad y cerrar cada paso con evidencia.

La idea central es esta: un subagente no es un reemplazo de pensamiento. Es una unidad de ejecución acotada que trabaja con contexto mínimo, propiedad clara y una salida verificable. Si falta alguna de esas piezas, conviene no delegar todavía.

Cuándo delegar

No todo merece un subagente. Delegar demasiado pronto genera ruido, duplicación y respuestas que parecen correctas pero no resuelven el problema. Delegar demasiado tarde te deja haciendo a mano tareas repetitivas que podrían haberse resuelto en paralelo.

Una buena regla práctica es delegar cuando se cumplen varias de estas condiciones:

- La tarea está bien delimitada.
- El resultado esperado se puede verificar.
- La tarea no necesita decisiones cruzadas con otras partes del sistema.
- Hay trabajo repetitivo, exploratorio o mecánico suficiente como para justificar el costo de coordinar.
- El contexto necesario cabe en unas pocas instrucciones y archivos concretos.
- El riesgo de que el subagente toque algo sensible está contenido por ownership o por read-only.

Ejemplos buenos de delegación:

- Buscar dónde está implementado un comportamiento.
- Mapear flujo de datos entre módulos.
- Identificar tests existentes para una zona del código.
- Redactar un documento de trabajo a partir de notas o una estructura previa.
- Ejecutar una validación puntual sobre una carpeta acotada.
- Probar hipótesis técnicas que no requieren editar muchas piezas a la vez.

Ejemplos malos de delegación:

- “Arreglá todo el sistema”.
- “Rehacé la arquitectura”.
- “Entrale al repo y mejorá lo que veas”.
- “Tomá decisiones de producto sin criterio de aceptación”.

- “Optimizá rendimiento” sin un punto de partida, una métrica y un alcance.

Si no podés decir qué archivo o qué salida querés, probablemente no estás listo para delegar.

Explorer y worker

La separación más útil en un flujo con subagentes es la de **explorer** y **worker**.

Explorer

El explorer sirve para entender. Su trabajo es leer, mapear, comparar y devolver información condensada. No debería hacer cambios salvo que la tarea sea explícitamente de exploración con anotación local, y aun así conviene mantenerlo read-only por defecto.

Usos típicos:

- Encontrar implementaciones.
- Seguir referencias.
- Resumir patrones existentes.
- Detectar tests, scripts o configuraciones relevantes.
- Comparar alternativas sin tocar el código.

Qué le pedís:

- que cite archivos concretos;
- que resuma con bullets cortos;
- que no proponga soluciones innecesarias;
- que identifique incertidumbres;
- que marque qué no pudo confirmar.

El explorer es especialmente valioso cuando no sabés todavía cuál es la superficie real del cambio. Antes de editar, primero entendé el mapa.

Worker

El worker sirve para hacer. Recibe un objetivo acotado, una zona de edición clara y criterios de salida verificables. Puede editar archivos, correr comandos o preparar un parche, pero siempre dentro de un ownership explícito.

Usos típicos:

- Implementar una función puntual.
- Crear un script de validación.
- Ajustar un archivo de documentación.
- Probar una hipótesis en una rama de trabajo o sobre un subconjunto de archivos.
- Preparar fixtures o datos de prueba.

Qué le pedís:

- que toque solo archivos autorizados;
- que no revierta cambios ajenos;
- que explique qué cambió;
- que valide con comandos concretos;
- que deje el trabajo listo para revisión.

La regla general es simple: explorer para reducir incertidumbre, worker para ejecutar una tarea ya entendida.

Roles cerrados en un harness

La separación explorer/worker es útil, pero un harness SDD necesita una división más estricta. La regla es: el rol que produce no debe ser el mismo rol que aprueba.

Una organizacion robusta puede usar cuatro roles:

Rol	Puede hacer	No puede hacer
leader	Orquestar, leer estado, lanzar agentes, cambiar estados	Implementar código
spec_author	Escribir requirements.md , design.md , tasks.md	Tocar src/ o tests/
implementer	Ejecutar tareas aprobadas, tocar código y tests	Autoaprobarse
reviewer	Revisar specs, tests, trazabilidad y checkpoints	Editar código

Entre spec_author e implementer debe existir una puerta de aprobación humana. El spec puede estar muy bien escrito y aun así resolver el problema equivocado. Por eso el implementador no arranca hasta que una persona acepta alcance, no objetivos y criterios de aceptación.

El leader mantiene el hilo principal. No debería convertirse en implementador cada vez que aparece una dificultad. Su valor está en mantener el proceso: una feature activa, spec aprobada, ownership claro, subagentes correctos y verificación final.

El spec_author convierte intención en contrato. Su salida no es una opinion por chat, sino archivos:

```
specs/<feature>/requirements.md
specs/<feature>/design.md
specs/<feature>/tasks.md
```

El implementer trabaja solo después de aprobación. Si el spec está propuesto o pendiente de aprobación, no implementa. Si el spec está aprobado, ejecuta tareas y deja evidencia.

El reviewer no arregla. Revisa. Si toca código, deja de ser reviewer y se rompe la separación de responsabilidades. Su salida debería ser un archivo de revisión:

Anti telefono descompuesto

Un problema común en sistemas con subagentes es que cada agente devuelve un resumen largo por chat. El siguiente agente lee ese resumen, no el artefacto original. Después otro resume el resumen. Así aparece el telefono descompuesto.

Regla operativa:

Los subagentes escriben resultados en archivos y devuelven solo una referencia.

Ejemplo:

```
spec_author:
  escribe specs/cli_recent/requirements.md
  escribe specs/cli_recent/design.md
  escribe specs/cli_recent/tasks.md
  responde: "Spec lista en specs/cli_recent/"

implementer:
  escribe progress/impl_cli_recent.md
  responde: "Implementación registrada en progress/impl_cli_recent.md"

reviewer:
  escribe progress/review_cli_recent.md
  responde: "Revisión aprobada/rechazada en progress/review_cli_recent.md"
```

El chat coordina. Los archivos conservan la verdad.

Ejemplo de lo que hay que evitar:

"El subagente dijo que estaba todo bien y que habia agregado tests."

Eso no alcanza. No dice que archivos cambio, que requirement cubrio, que comando corrio ni que evidencia dejo.

Formato mínimo de salida aceptable:

```
Resultado: implementado
Artefacto: progress/impl_cli_recent.md
Archivos tocados: src/cli.py, tests/test_cli.py
Evidencia: `./init.sh` verde, 27 tests passed
Bloqueos: ninguno
```

Contrato de handoff:

Rol	Entrada permitida	Salida esperada	Cuándo escalar
explorer	Pregunta acotada, rutas, criterio de búsqueda	Hallazgos con evidencia y dudas abiertas	Si falta información o aparecen contradicciones
spec_author	Issue, contexto, no objetivos	Requirements, design y tasks trazables	Si el alcance no puede cerrarse sin decisión humana
implementer	Spec aprobada, ownership y tests esperados	Cambio acotado, archivos tocados y evidencia	Si necesita tocar fuera de ownership
reviewer	Diff, spec, tests y trace	Hallazgos, bloqueo o aprobación razonada	Si el resultado contradice el contrato

Si un subagente devuelve algo parcial, no se lo resume para seguir igual. Se decide una de tres cosas: pedir aclaración, reintentar con un recorte más chico o escalar a decisión humana. Esa regla evita integrar trabajo ambiguo por inercia.

Contexto mínimo

Uno de los errores más comunes al delegar es pasar demasiado contexto. Darle a un subagente “todo lo que hay” parece seguro, pero suele degradar la calidad. A mayor contexto, más ruido, más costo y más chances de que el agente mezcle señales irrelevantes.

El buen contexto mínimo responde estas preguntas:

1. Qué querés lograr.
2. Qué parte del sistema puede tocar.
3. Qué archivos o rutas son relevantes.
4. Qué no debe tocar.
5. Cómo sabés si terminó bien.

Un encuadre útil puede tener esta forma:

- Objetivo: una frase concreta.
- Alcance: archivos y carpetas.
- Restricciones: no tocar X, no modificar Y, no cambiar comportamiento fuera de Z.
- Salida esperada: resumen, parche, lista de hallazgos o script.
- Verificación: tests, comandos o revisión manual.

No hace falta explicar toda la historia del proyecto. Hace falta explicar la porción que el subagente necesita para actuar sin improvisar.

Qué incluir

- El problema exacto.
- El formato de la salida.
- Los archivos que marcan ownership.
- Los comandos de validación.
- Los criterios de aceptación.

Qué evitar

- Historias largas sin relevancia operativa.
- Opiniones contradictorias.
- Pistas viejas que ya no aplican.
- Capturas mentales que el agente no puede verificar.
- Instrucciones abiertas como "mejoralo bastante".

Si el subagente necesita una aclaración clave para no errar, mejor frenar y redefinir. Si solo necesita detalles ruidosos, no los pases.

Ownership de archivos

El ownership es lo que evita que varios agentes pisen el mismo terreno. Cada tarea debe tener una frontera clara: qué archivos puede editar un worker, qué archivos solo puede leer, y qué partes del sistema están fuera de juego.

Esto no es solo higiene. Es una forma de preservar integridad mientras se trabaja en paralelo.

Una asignación bien hecha incluye:

- archivo o carpeta autorizada;
- tipo de permiso: lectura, edición o solo inspección;
- límites de impacto;
- criterio para considerar invasivo un cambio;
- confirmación de que no debe revertir trabajo ajeno.

Ejemplo de ownership bueno:

- Worker A: `src/docs/intro.md` y `src/docs/glosario.md`, solo edición de contenido.
- Worker B: `scripts/validate-docs.mjs`, solo este archivo y su test asociado.
- Explorer C: cualquier archivo bajo `src/`, pero sin editar.

Ejemplo de ownership malo:

- "Editá lo que haga falta".
- "Acomodá todo el módulo".
- "Mirá si encontrás algo mejor".

Cuando el ownership está claro, la revisión también mejora. Sabés qué cambió, por qué cambió y qué queda fuera del scope.

Paralelismo

Los subagentes brillan cuando podés dividir trabajo independiente. El paralelismo no significa hacer más cosas al mismo tiempo por ansiedad, sino separar tareas que no se bloquean entre sí.

Hay tres niveles útiles:

Paralelismo de exploración

Varios explorers buscan información distinta en paralelo.

Ejemplo:

- uno localiza la implementación;
- otro identifica tests;
- otro resume patrones similares;
- otro busca riesgos o dependencias.

Esto sirve mucho al inicio de una tarea grande. En vez de leer todo secuencialmente, obtenés un mapa más rápido.

Paralelismo de ejecución

Varios workers hacen cambios en áreas distintas, siempre que los archivos no se solapen.

Ejemplo:

- uno edita documentación;
- otro ajusta una validación;
- otro prepara ejemplos o fixtures.

Esto requiere disciplina. Si dos workers comparten archivos, el supuesto de paralelismo se rompe y empezás a competir con merges o reescrituras.

Paralelismo de verificación

Un subagente implementa y otro verifica desde afuera.

Ejemplo:

- worker A cambia una función;
- worker B revisa que los tests relevantes existan y que el cambio no rompa convenciones;
- el hilo principal integra el resultado.

Este patrón es útil para separar producción de evidencia. La verificación independiente reduce sesgo de confirmación.

Cuándo no paralelizar

No conviene paralelizar cuando:

- la decisión de una tarea depende del resultado de otra;
- el mismo archivo será editado por varios agentes;
- la arquitectura todavía está en discusión;
- el costo de coordinar supera el ahorro;
- un error podría contaminar varias piezas a la vez.

Paralelizar no es un objetivo en sí. Es una herramienta cuando la independencia existe de verdad.

Anti-patrones

Los anti-patrones suelen verse como productividad al principio y como deuda operativa después.

1. Delegación vaga

Pedís algo demasiado amplio y obtenés algo demasiado genérico. El agente rellena huecos con suposiciones.

Señal típica: la respuesta suena prolija pero no aterriza en archivos ni decisiones concretas.

2. Context dumping

Le pasás todo el repo, todo el chat, todas las notas. El agente pierde foco.

Señal típica: respuestas largas con fragmentos relevantes mezclados con ruido.

3. Ownership difuso

Nadie sabe quién puede tocar qué. Aparecen ediciones cruzadas, pisadas y confusión.

Señal típica: "yo pensé que esa carpeta era libre".

4. Subagentes que resuelven diseño

El subagente empieza a inventar estrategia cuando solo tenía que ejecutar una slice concreta.

Señal típica: propone reestructurar todo antes de terminar el cambio pedido.

5. Paralelismo falso

Se lanzan varias tareas "en paralelo" que en realidad compiten por la misma zona.

Señal típica: conflicto de archivos, resultados inconsistentes o necesidad de rehacer trabajo.

6. Cierre sin evidencia

El subagente dice que terminó, pero no muestra cómo se valida.

Señal típica: no hay comando, no hay archivo, no hay criterio de aceptación.

7. Mezclar lectura con edición sin control

Un agente explora y además toca cosas fuera de scope porque "ya que estaba".

Señal típica: cambios colaterales no pedidos.

La regla de oro es dura pero simple: si una tarea no se puede revisar con claridad, probablemente fue delegada mal.

Matriz de modelos y esfuerzo

No todos los subagentes necesitan el mismo tipo de modelo ni el mismo nivel de razonamiento. Conviene pensar en una matriz práctica: complejidad de la tarea por esfuerzo requerido.

La idea no es memorizar nombres, sino usar una combinación razonable según el trabajo.

Tareas mecánicas

- Ejemplos: inspección de archivos, búsqueda, extracción de patrones, validaciones simples, formateo.
- Modelo sugerido: uno liviano o rápido.
- Esfuerzo: bajo.
- Objetivo: velocidad y bajo costo.

Tareas de síntesis

- Ejemplos: resumir hallazgos, comparar opciones, condensar documentación, mapear flujo.
- Modelo sugerido: liviano o medio, según la densidad del material.
- Esfuerzo: bajo a medio.
- Objetivo: ordenar información sin sobredimensionar la respuesta.

Tareas de implementación acotada

- Ejemplos: editar un archivo, crear un script, ajustar una prueba.
- Modelo sugerido: medio.
- Esfuerzo: medio.
- Objetivo: buen criterio técnico sin costo excesivo.

Tareas de debugging complejo

- Ejemplos: bugs con múltiples causas, integración cruzada, fallas intermitentes.
- Modelo sugerido: uno más fuerte.
- Esfuerzo: medio a alto.
- Objetivo: más capacidad de razonamiento, menos atajos.

Tareas de revisión final

- Ejemplos: revisar un parche importante, detectar regresiones, cuestionar supuestos.
- Modelo sugerido: más fuerte o al menos distinto al que implementó.

- Esfuerzo: medio a alto.
- Objetivo: independencia y mirada crítica.

Regla práctica

- Si la tarea es repetitiva, no gastes un modelo pesado.
- Si la tarea depende de criterios finos o cruza varias piezas, subí el nivel.
- Si el resultado va a decidir una entrega importante, agregá revisión independiente.

El esfuerzo no debe ser “siempre alto”. Tiene que acompañar el riesgo y la ambigüedad.

Ejemplos de prompts de delegación

Los mejores prompts de subagente no parecen pedidos abiertos. Parecen tickets bien escritos.

Exploración de implementación

```
Objetivo: encontrar dónde se implementa el flujo de guardado automático.
Rol: explorer.
Alcance: solo lectura sobre `src/` y `tests/`.
Entrega: lista de archivos relevantes, resumen breve del flujo y riesgos.
No hagas cambios.
Si algo no está claro, marcá la incertidumbre.
```

Mapeo de tests

```
Objetivo: identificar tests existentes para la validación de rutas.
Rol: explorer.
Alcance: buscar en `tests/`, `spec/` y scripts de CI.
Entrega: tabla simple con archivo, propósito y cobertura.
No edites nada.
```

Implementación acotada

```
Objetivo: agregar un helper para normalizar títulos en `src/utils/title.js`.
Rol: worker.
Ownership exclusivo: solo `src/utils/title.js`.
Restricciones: no tocar otros archivos, no cambiar interfaces públicas.
Entrega: código final, explicación breve y comando de verificación.
```

Documento técnico

```
Objetivo: redactar un documento técnico sobre uso de subagentes.
Rol: worker.
Ownership: solo `src/02-subagentes-inteligentes.md`.
Estilo: claro, serio, accionable, en español rioplatense neutro.
```


Incluí criterios, anti-patrones, ejemplos y checklist de cierre.
No generes archivos extra.

Verificación independiente

Objetivo: revisar el cambio y buscar errores lógicos o scope creep.
Rol: explorer.
Alcance: leer el parche y los archivos tocados.
Entrega: hallazgos concretos, dudas abiertas y sugerencias de validación.
No modifiques archivos.

Fijate que en todos los casos el prompt define rol, alcance, entrega y límites. Eso reduce ambigüedad y mejora la calidad de salida.

Checklist de cierre

Una delegación no termina cuando el subagente responde. Termina cuando el resultado quedó verificado y el hilo principal puede seguir sin arrastrar dudas.

Checklist de cierre para usar siempre:

- El objetivo quedó cumplido en una frase verificable.
- El subagente trabajó dentro del ownership autorizado.
- No hubo cambios fuera de scope.
- Los archivos tocados quedaron identificados.
- La salida es reproducible o revisable.
- Si hubo cambios de código, existe comando de validación.
- Si hubo cambios de documento, el contenido cumple la estructura pedida.
- Las incertidumbres quedaron explícitas.
- No hay conflictos con trabajo paralelo.
- El resultado final fue leído por el hilo principal antes de considerarlo cerrado.

Una versión más estricta para tareas sensibles:

- Revisé el diff.
- Verifiqué que no se tocaron archivos ajenos.
- Corrí el comando de validación pertinente.
- Confirmé que no hay supuestos ocultos.
- Registré qué queda pendiente, si algo quedó afuera.

Proceso recomendado

Un flujo robusto para usar subagentes suele verse así:

1. Definís la tarea y su criterio de salida.
2. Separás lo que es exploración de lo que es ejecución.
3. Asignás ownership explícito.
4. Elegís el nivel de modelo y esfuerzo según ambigüedad y riesgo.
5. Corrés tareas en paralelo solo si no se pisan.
6. Consolidás resultados en el hilo principal.
7. Verificás el cierre con evidencia.

Si la tarea es grande, este flujo se puede repetir por capas: primero explorers, después workers, después revisión independiente.

Señales de que vas bien

Vas bien cuando:

- el subagente responde con menos texto pero más precisión;
- cada salida menciona archivos o comandos concretos;
- el hilo principal decide más rápido porque recibió información mejor filtrada;
- el paralelismo reduce tiempo sin aumentar retrabajo;
- los errores se detectan antes de integrar.

Vas mal cuando:

- tenés que reinterpretar todo lo que devolvieron;
- aparecen cambios sorpresa;
- el contexto crece sin control;
- la verificación se hace recién al final y con sorpresas;
- cada subagente necesita que le expliques de nuevo la misma base.

Cierre operativo

Usar subagentes con inteligencia no es una cuestión de cantidad, sino de forma. La calidad de la delegación depende más del recorte, la propiedad y la verificación que del número de agentes en juego.

Si necesitás explorar, usá explorer. Si necesitás ejecutar, usá worker. Si necesitás varias cosas a la vez, dividí el trabajo de forma que no se pisen. Y si no podés definir un ownership claro, todavía no conviene delegar.

La mejor práctica no es "hacer que la IA trabaje sola". Es armar una cadena de trabajo donde cada pieza tenga un rol acotado, una salida comprobable y un cierre explícito. Ahí es cuando los subagentes dejan de ser una promesa y pasan a ser una herramienta útil de verdad.

SDD y Estructura de Soporte

Specification-Driven Development aplicado a repositorios con IA

Al terminar este volumen podés armar una estructura SDD mantenible para repositorios asistidos por IA.

SDD, o *Specification-Driven Development*, es una forma de trabajar en la que la especificación no es un documento decorativo ni una nota aislada: es la pieza central del flujo de desarrollo. En lugar de arrancar por código suelto, arrancamos por una definición clara de lo que se quiere construir, por qué existe, cómo se va a validar y qué decisiones quedan registradas en el camino.

En un repositorio moderno, y más todavía cuando hay IA involucrada, SDD ordena el trabajo en capas. Primero se define el problema. Después se convierte ese problema en una especificación verificable. Luego se descompone en tareas. Recién ahí se implementa. Y al final se deja trazabilidad: qué se cambió, qué se descartó, qué se probó y qué quedó listo para operar.

Esto sirve especialmente cuando el equipo usa IA como copiloto, como generadora de propuestas o como asistente de análisis. La IA puede acelerar muchísimo, pero también puede inventar supuestos, mezclar prioridades o producir código que funciona sin respetar el contexto. SDD le pone límites útiles a eso. La IA no adivina el objetivo: lo lee. No improvisa criterios de aceptación: los sigue. No reemplaza decisiones: las documenta o las propone para aprobación.

Qué resuelve SDD

SDD resuelve un problema muy concreto: la distancia entre la intención y la implementación. En repositorios grandes, esa distancia suele crecer en silencio. Un issue dice una cosa, un PR resuelve otra, el código termina haciendo algo más, y nadie sabe cuál era la versión correcta de la idea.

Con SDD, esa cadena queda explícita:

- **requirements** : qué necesidad existe y a quién le importa.
- **specs** : cómo se describe el comportamiento esperado.
- **decisions** : qué alternativas se evaluaron y cuál se eligió.
- **tasks** : qué pasos concretos hay que ejecutar.
- **acceptance criteria** : cómo se sabe que quedó bien.
- **traces** : qué evidencia conecta la spec con el código y las pruebas.
- **runbooks** : cómo operarlo o recuperarlo cuando falla.
- **ADRs** : decisiones de arquitectura con contexto y consecuencias.
- **evaluation notes** : resultados de validaciones, pruebas asistidas o análisis comparativos.

La ventaja no es solo documental. También mejora la ejecución. Cuando la estructura es buena, la IA genera menos basura, el equipo discute mejor y los cambios son más fáciles de revisar.

Principio base

La regla principal de SDD es simple: cada pieza debe tener un dueño semántico.

- El requerimiento explica el problema.
- La spec define el comportamiento.
- La decisión explica el trade-off.
- La tarea ordena el trabajo.

- La aceptación cierra el alcance.
- La traza permite seguir el hilo.
- El runbook prepara la operación.

Si todo está mezclado en un solo archivo, el sistema se vuelve frágil. Si está demasiado atomizado sin criterio, también. SDD busca un equilibrio: separar lo suficiente para que cada cosa tenga función propia, pero no tanto como para que entender una feature requiera abrir veinte archivos sin relación.

SDD estricto y puerta humana

En una versión estricta, SDD no significa "escribir una idea antes de codear". Significa que el agente no implementa hasta que existe un contrato aprobado.

Flujo recomendado:

```
pending
-> spec_author
-> spec_ready
-> HUMANO APRUEBA
-> in_progress
-> implementer
-> reviewer
-> done
```

La puerta humana es deliberada. La IA puede escribir un spec muy convincente y aun así equivocarse en el objetivo de negocio. Por eso, antes de tocar código, alguien debe aprobar explícitamente:

```
aprobado
```

Si no hay aprobación, no hay implementación. El costo de esperar esa confirmación es menor que el costo de corregir una feature bien escrita pero mal entendida.

Formato mínimo de aprobación:

```
Estado: aprobado
Aprobado por: <nombre o rol>
Fecha: <YYYY-MM-DD>
Alcance aprobado: R1, R2, R3
No objetivos aceptados: <lista breve>
Artefactos autorizados: requirements.md, design.md, tasks.md
Condición de cierre: todos los R<n> con test y `init.sh` verde
```

Si cambia el alcance después de aprobar, el estado vuelve a revisión. Esa regla evita que una implementación crezca por inercia mientras el spec queda viejo.

Requirements en EARS

Para specs estrictas conviene usar EARS: **Easy Approach to Requirements Syntax**. No es una ceremonia; es una forma de escribir requirements verificables.

Patrones base:

Patrón	Forma
Ubicuo	El sistema DEBE <comportamiento>.
Evento	CUANDO <evento>, el sistema DEBE <comportamiento>.
Estado	MIENTRAS <estado>, el sistema DEBE <comportamiento>.
Opcional	DONDE <condicion>, el sistema DEBE <comportamiento>.
No deseado	SI <situación no deseada> ENTONCES el sistema DEBE <respuesta>.

Ejemplo:

```
## R1

CUANDO el usuario ejecuta `notes recent`, el sistema DEBE mostrar hasta 5 notas ordenadas por fecha descendente.

## R2

SI `--limit` recibe un valor menor o igual a 0 ENTONCES el sistema DEBE mostrar un error y salir con código distinto de 0.
```

Reglas:

1. Cada requirement tiene un id estable: **R1**, **R2**, **R3**.
2. Cada requirement debe ser verificable.
3. Un requirement no debe mezclar varios **DEBE**.
4. Evitar verbos blandos como "podría", "soporta" o "intenta".
5. Cada **R<n>** debe mapear a por lo menos un test.

Ejemplo de descomposicion:

```
Malo:

"El sistema debería permitir ver notas recientes rápido, con límite configurable y buen manejo de errores."

Bueno:

R1: CUANDO el usuario ejecuta `notes recent`, el sistema DEBE mostrar notas ordenadas por fecha descendente.
R2: DONDE el usuario pasa `--limit`, el sistema DEBE limitar la cantidad de notas al valor indicado.
R3: SI `--limit` recibe un valor inválido ENTONCES el sistema DEBE mostrar un error y salir con código distinto de 0.
```

Design y tasks trazables

El `design.md` hace visibles las decisiones antes de editar archivos. Debe decir que se toca, que se descarta, que riesgos existen y que no entra en alcance.

Plantilla mínima:

```
# Design - <feature>

## Archivos afectados

- `src/...`
- `tests/...`

## Decisiones

1. ...
2. ...

## Alternativa descartada

Se descarta <opción> porque <motivo>.

## Riesgos

- ...

## No objetivos

- ...
```

`tasks.md` debe ser ejecutable y trazable:

```
- [ ] T1 - Agregar `cmd_recent` en `src/cli.py`. Cubre: R1.
- [ ] T2 - Registrar flag `--limit`. Cubre: R1, R2.
- [ ] T3 - Agregar `test_recent_default_limit`. Cubre: R1.
- [ ] T4 - Agregar `test_recent_invalid_limit`. Cubre: R2.
```

El reviewer debería rechazar una feature si queda un requirement sin test, una task sin requirement o un spec aprobado que no coincide con el código.

Ejemplo de trace completo:

```
# TRACE-014: notas recientes

Issue: #14
Spec: `specs/cli_recent/requirements.md`
Design: `specs/cli_recent/design.md`
Tasks: `specs/cli_recent/tasks.md`
```

PR: #22

```
| Requirement | Task | Test | Evidencia |  
| --- | --- | --- | --- |  
| R1 | T1, T3 | `test_recent_default_limit` | lista ordenada por fecha descendente |  
| R2 | T2, T4 | `test_recent_custom_limit` | respeta `--limit` |  
| R3 | T5 | `test_recent_invalid_limit` | error y exit code distinto de 0 |
```

Resultado de cierre:

```
- `./init.sh`: verde  
- tests: 27 passed  
- reviewer: aprobado
```

Carpeta recomendada

La convención sugerida para este volumen es concentrar el material de SDD alrededor de [.github/orquestador/sdd](https://github.com/orquestador/sdd). Esa carpeta funciona como núcleo de gobernanza y ejecución para specs, decisiones y trabajo asistido por IA.

Una estructura posible es esta:

```
.github/orquestador/sdd/  
  README.md  
  index.md  
  requirements/  
    _plantilla.md  
    <area>-<id>.md  
  specs/  
    _plantilla.md  
    <feature>-<id>.md  
  decisions/  
    _plantilla-adr.md  
    <adr>-<id>.md  
  tasks/  
    _plantilla.md  
    <issue>-<id>.md  
  acceptance/  
    _plantilla.md  
    <feature>-<id>.md  
  traces/  
    _plantilla.md  
    <feature>-<id>.md  
  runbooks/  
    _plantilla.md  
    <system>-<id>.md  
  evaluations/  
    _plantilla.md  
    <feature>-<id>.md  
  progress/
```



```
current.md
history.md
examples/
sample-issue.md
sample-spec.md
```

No hace falta que todas las carpetas existan desde el día uno. La base práctica suele ser `requirements/`, `specs/`, `tasks/`, `acceptance/` y `traces/`. `decisions/`, `runbooks/`, `evaluations/`, `progress/` y `examples/` se agregan cuando el repositorio ya los necesita. Lo importante es que la arquitectura esté pensada para escalar sin perder legibilidad.

Qué va en cada carpeta

README.md

Es la puerta de entrada. Debe decir qué es SDD en este repositorio, cuál es el objetivo de la carpeta y cómo se usa. No debería contener la teoría completa, sino una guía corta para navegar el sistema.

Contenido esperado:

- propósito de la carpeta;
- convención de nombres;
- orden recomendado de lectura;
- cómo crear una nueva spec o una nueva decisión;
- relación con issues, PRs y documentación general.

index.md

Es el mapa de navegación. Lista las specs activas, los ADRs relevantes, las evaluaciones más recientes y los runbooks críticos. También puede incluir estado: propuesto, en revisión, aprobado, implementado, obsoleto.

Contenido esperado:

- catálogo de artefactos vivos;
- links cruzados;
- estado por documento;
- fecha de última actualización;
- responsable o área.

requirements/

Acá viven los requerimientos. No son tickets técnicos todavía. Son necesidades, dolores, objetivos o restricciones del negocio, producto u operación.

Un requerimiento bien escrito responde:

- qué problema existe;
- para quién;

- qué pasa si no se resuelve;
- qué restricciones hay;
- qué señales indicarían éxito.

Ejemplo breve:

```
# REQ-014: reducir errores en la alta de clientes
```

Problema: hoy los formularios permiten guardar datos incompletos y eso genera retrabajo.

Objetivo: bloquear altas inválidas antes de persistir.

Restricciones: no romper el flujo actual de edición.

Impacto: menos soporte manual y menos errores en reportes.

specs/

La spec describe el comportamiento. Ya no habla solo del problema, sino del sistema esperado. Tiene que ser verificable. Una buena spec deja en claro entradas, salidas, reglas, estados, errores, permisos y escenarios límite.

Contenido esperado:

- contexto;
- alcance;
- comportamiento principal;
- casos borde;
- flujos alternativos;
- dependencias;
- criterios de aceptación;
- riesgos;
- supuestos explícitos.

La spec no debería escribir código, pero sí puede incluir pseudo-reglas, ejemplos de payload, tablas de estados o secuencias.

decisions/

Acá viven los ADRs y cualquier decisión estructural que valga la pena recordar. Si la spec dice *qué* hacer, la decisión explica *por qué* se eligió ese camino y no otro.

Contenido esperado:

- contexto;
- opciones consideradas;
- decisión tomada;
- consecuencias;
- trade-offs;
- fecha;

- autor o equipo.

Ejemplo de decisión:

```
# ADR-007: validar en servidor y no solo en cliente
```

Se elige validar en backend como fuente de verdad.

Razón: el cliente puede quedar desactualizado y no es confiable como única barrera.

Consecuencia: se duplica parte de la lógica, pero se reduce el riesgo operativo.

tasks/

Las tasks descomponen la implementación en pasos chicos y verificables. Son el puente entre la spec y el código. Una task buena no describe una aspiración vaga, sino una acción concreta.

Contenido esperado:

- objetivo de la task;
- dependencias;
- archivos o módulos involucrados;
- criterio de terminado;
- evidencia esperada.

Una task mal escrita dice: "hacer la feature". Una task útil dice: "agregar validación del campo X en el endpoint Y y cubrirlo con test de integración".

acceptance/

Esta carpeta contiene los criterios de aceptación. Su función es cerrar el contrato entre lo pedido y lo entregado. Tiene que ser lo bastante concreta para que un reviewer o una IA puedan chequearla sin inventar interpretación.

Contenido esperado:

- condiciones de pase/falla;
- escenarios nominales;
- errores esperados;
- comportamiento observable;
- requisitos no funcionales si aplican.

traces/

Las traces conectan el requerimiento, la spec, las tareas y el resultado final. Sirven para saber de dónde salió cada decisión y qué evidencia la respalda.

Contenido esperado:

- vínculo `REQ -> SPEC -> TASK -> PR -> TEST` ;
- resumen de cobertura;
- referencias a commits, issues o PRs;

- notas sobre lo que quedó fuera.

runbooks/

Los runbooks documentan cómo operar o recuperar un flujo cuando algo falla. Son especialmente útiles cuando una spec termina en una feature con impacto real: pagos, autenticación, jobs, sincronizaciones, migraciones o procesos automáticos.

Contenido esperado:

- síntomas comunes;
- pasos de diagnóstico;
- pasos de recuperación;
- rollback si aplica;
- señales de alerta;
- contactos o dependencias.

evaluations/

Acá van las notas de evaluación: pruebas humanas, pruebas con IA, benchmarks internos, comparaciones antes/después, análisis de riesgo o revisiones de calidad.

Contenido esperado:

- qué se evaluó;
- con qué método;
- qué resultado dio;
- qué defectos aparecieron;
- qué decisión se tomó a partir de eso.

progress/

`progress/` no reemplaza a SDD. Es el diario operativo de una sesión o una feature en curso. Sirve para que un humano o un agente puedan retomar sin depender del chat anterior.

Convención mínima:

```
progress/current.md -> estado vivo, bloqueos, siguiente paso
progress/history.md -> cierres, evidencias y decisiones ya tomadas
```

Si el equipo no necesita este nivel de continuidad, puede omitirlo. Si trabaja con agentes durante varias sesiones, suele ahorrar mucho contexto.

examples/

Sirve para bajar fricción. Tener plantillas y ejemplos concretos mejora mucho la adopción, sobre todo si el equipo recién empieza a usar SDD.

Contenido esperado:

- issue ejemplo;
- spec ejemplo;
- ADR ejemplo;
- task ejemplo;
- checklist ejemplo.

Ciclo de trabajo de una issue o feature

Un flujo SDD sano no arranca en el PR. Arranca antes, cuando todavía se puede corregir el rumbo barato.

1. Abrir el requerimiento

Primero se redacta el problema. No hace falta poesía. Hace falta precisión. El requerimiento debe explicar por qué existe la iniciativa y qué dolor resuelve.

2. Convertirlo en spec

Después se define el comportamiento esperado. Acá se responde:

- qué entra;
- qué sale;
- qué reglas aplican;
- qué errores se permiten;
- qué no entra en esta versión.

3. Registrar decisiones

Si hay alternativas relevantes, se deja un ADR. Esto evita que, dentro de dos meses, alguien vuelva a discutir una decisión ya tomada como si nada hubiera pasado.

4. Dividir en tasks

La implementación se parte en tareas chicas. Cada task debería poder hacerse, revisarse y testearse de forma independiente.

5. Definir aceptación

Antes de tocar código, los criterios de aceptación ya tienen que estar escritos. Si se escriben después, suelen adaptarse al resultado y pierden valor.

6. Implementar con trazabilidad

Cada cambio debe dejar conexión con la spec. Eso puede ser mediante referencias en el PR, notas en la task o vínculos en la trace. Lo importante es que el hilo no se corte.

7. Evaluar

Al final se registra lo que pasó: pruebas realizadas, problemas detectados, coberturas faltantes, deuda pendiente y decisión final.

Un ejemplo de ciclo completo:

1. **REQ-014** detecta errores en la alta de clientes.
2. **SPEC-014** define validaciones, mensajes y estados.
3. **ADR-007** elige validar en backend.
4. **TASK-014.1** agrega validaciones.
5. **TASK-014.2** cubre tests de integración.
6. **AC-014** define 5 criterios de aceptación.
7. **TRACE-014** enlaza issue, PR y pruebas.
8. **EVAL-014** resume el resultado y puntos abiertos.

Plantillas cortas

Requerimiento

```
# REQ-XXX: título corto
```

Problema:

Objetivo:

Usuarios impactados:

Restricciones:

Riesgo de no hacerlo:

Spec

```
# SPEC-XXX: título corto
```

Contexto:

Alcance:

Fuera de alcance:

Comportamiento esperado:

Casos borde:

Criterios de aceptación:

ADR

```
# ADR-XXX: decisión corta
```

Contexto:

Opciones:

Decisión:

Consecuencias:

Task

TASK-XXX: acción concreta

Qué hay que hacer:

Archivos probables:

Dependencias:

Cómo se valida:

Acceptance

AC-XXX: feature o spec

- Dado ...
- Cuando ...
- Entonces ...

Trace

TRACE-XXX: título corto

REQ:

SPEC:

TASKS:

PR:

TESTS:

NOTAS:

Runbook

RUN-XXX: sistema o flujo

Síntoma:

Diagnóstico:

Recuperación:

Rollback:

Evaluation notes

EVAL-XXX: título corto

Qué se probó:

Criterio:

Resultado:

Problemas:

Decisión:

Cómo usar SDD con IA

La IA cambia el valor de la documentación. Antes, un spec servía sobre todo para humanos. Ahora también sirve como contrato de contexto para asistentes.

Un buen flujo con IA es este:

- la IA lee el requerimiento y resume el problema;
- la IA propone una spec inicial;
- el humano valida alcance y prioridades;
- la IA descompone tasks y sugiere tests;
- el humano aprueba o corrige decisiones;
- la IA ayuda a implementar y a revisar trazabilidad;
- el humano verifica aceptación final.

Eso funciona mejor cuando el repositorio tiene artefactos estables y legibles. Si la spec está desordenada, la IA va a producir respuestas desordenadas. Si la aceptación es vaga, la IA va a completar los huecos con supuestos. Si no hay ADRs, las decisiones se pierden y el contexto se reinicia cada vez.

La clave es usar la IA para acelerar razonamiento, no para reemplazarlo. SDD hace que la IA trabaje sobre una base explícita. En vez de “haceme una feature”, se le pide “tomando esta spec y estas acceptance criteria, desglosá la implementación y marcá riesgos”. Ese cambio de lenguaje mejora mucho la calidad del resultado.

Anti-patrones

Mezclar todo en un solo archivo

Cuando requirements, spec, task y decisión viven juntos sin estructura, el documento se vuelve imposible de usar. Nadie sabe qué parte es fuente de verdad.

Especificaciones ambiguas

Frases como “debe ser intuitivo” o “debe andar bien” no sirven si no se traducen en criterios verificables. Si no se puede probar, está incompleto.

Tasks demasiado grandes

Una task que dice “implementar todo el flujo” suele terminar en una PR difícil de revisar y en una deuda de trazabilidad.

ADRs a destiempo

Escribir decisiones después de haber olvidado el contexto destruye su valor. Un ADR sirve cuando conserva la discusión real.

Acceptance reescrita para que cierre

Si los criterios de aceptación se ajustan al resultado del código, dejan de ser un contrato y pasan a ser una justificación.

Traces vacías

No alcanza con decir “está en el PR”. Si no se puede seguir el hilo hasta el requerimiento original, la trazabilidad no existe.

Runbooks fantasmas

Un runbook que nadie puede ejecutar en una emergencia no sirve. Tiene que ser corto, claro y accionable.

Evaluaciones sin conclusión

Medir algo y no decir qué decisión se tomó es trabajo incompleto. La evaluación tiene que dejar una postura concreta.

Ejemplo end-to-end

Feature: comando `recent` para listar notas recientes.

```
issue-014.md
  Problema: el usuario no puede ver notas recientes rápido.

requirements/recent-notes.md
  R1: mostrar las 10 notas más recientes por defecto.
  R2: permitir cambiar el límite con --limit.
  R3: devolver lista vacía sin error si no hay notas.

specs/recent-notes.md
  Comando: notes recent [--limit n]
  Orden: updated_at descendente.
  Límite máximo: 50.

tasks/recent-notes.md
  T1 -> R1: agregar query ordenada.
  T2 -> R2: validar --limit.
  T3 -> R3: cubrir store vacío.

acceptance/recent-notes.md
  A1: con 12 notas muestra 10.
  A2: con --limit 5 muestra 5.
  A3: sin notas muestra mensaje neutro.

traces/recent-notes.md
  R1 -> T1 -> test_recent_default_limit
  R2 -> T2 -> test_recent_custom_limit
  R3 -> T3 -> test_recent_empty_store
```

Regla de mantenimiento: si cambia un requirement, primero se actualiza `requirements/`, después `specs/`, luego `tasks/` y finalmente `traces/`. El reviewer bloquea el cierre si el código cambió pero la cadena requirement-task-test quedó vieja.

Checklist práctico

Antes de cerrar una feature bajo SDD, conviene revisar esto:

- el requerimiento está escrito y tiene problema, objetivo e impacto;
- la spec define comportamiento verificable;
- el alcance excluye lo que no entra;
- las decisiones relevantes están documentadas;
- las tasks están partidas en pasos chicos;
- los criterios de aceptación son concretos;
- hay trazabilidad entre issue, spec, PR y tests;
- existe runbook si el flujo afecta operación;
- quedaron notas de evaluación o validación;
- no hay contradicciones entre documento, código y pruebas;
- el repositorio puede reconstruir el contexto sin depender de memoria oral.

Recomendación de mantenimiento

Para que esta estructura no se vuelva burocrática, conviene sostener tres reglas:

1. escribir poco pero útil;
2. actualizar al mismo ritmo que el código;
3. cerrar cada feature con evidencia.

No se trata de documentar por documentar. Se trata de que la documentación funcione como infraestructura de trabajo. En un repo con IA, esa infraestructura es parte del producto. Reduce errores, mejora la revisión y hace que el conocimiento sobreviva a la rotación de contexto.

Cierre

SDD no es una moda de naming ni un reemplazo mágico de ingeniería. Es una disciplina para ordenar la intención antes de que el código la opaque. En repositorios donde la IA participa en la escritura, la revisión o el análisis, esta disciplina importa todavía más, porque el costo de la ambigüedad se multiplica.

La estructura de `.github/orquestador/sdd` propone algo simple: separar las piezas correctas para que cada una cumpla su función y todas juntas formen un sistema legible. Requirements, specs, decisions, tasks, acceptance criteria, trases, runbooks, ADRs y evaluation notes no son carpetas decorativas. Son el mecanismo para que una idea pase de problema a implementación sin perder contexto en el camino.

Si el repositorio adopta esta forma de trabajo, cada feature deja de ser un salto de fe. Pasa a ser un proceso rastreable, revisable y mejorable. Y eso, en un stack asistido por IA, vale más que cualquier atajo improvisado.

VOLUMEN 04

AGENTS.md, .github y Comandos Estilo Prompt

Cómo construir un repositorio preparado para agentes, SDD, Copilot, workflows y prompts reutilizables

Al terminar este volumen podés ordenar AGENTS.md, .github, prompts reutilizables y gates de validación.

Para que existe este tomo

Un repositorio moderno no debería depender de que el humano recuerde todas las reglas cada vez que abre un asistente de IA. Las reglas del proyecto tienen que vivir en archivos. Algunas reglas son para humanos, otras para agentes, otras para Copilot, otras para CI y otras para el proceso SDD. Cuando esas capas están mezcladas, la IA trabaja con contexto incompleto y el equipo termina corrigiendo los mismos errores una y otra vez.

Este tomo propone una estructura práctica para repositorios que quieren usar IA de forma seria:

1. `AGENTS.md` como contrato operativo para agentes de código.
2. `.github/copilot-instructions.md` como instrucciones generales para Copilot.
3. `.github/instructions/*.instructions.md` como instrucciones específicas por path.
4. `.github/prompts/*.prompt.md` como comandos reutilizables estilo prompt.
5. `.github/orquestador/sdd/*` como sistema de especificaciones, decisiones y trazabilidad.
6. `.github/workflows/*` como gate remoto principal cuando GitHub es la superficie de trabajo.
7. `.github/orquestador/pipelines/catalog.md` como catálogo de gobierno, no como segundo motor de automatización.

La idea no es llenar el repo de ceremonias. La idea es que cada archivo tenga un propósito claro y que un agente pueda responder tres preguntas rápido: qué proyecto es este, cómo se trabaja acá y cómo se verifica que el cambio está bien.

Mapa de responsabilidades

Archivo o carpeta	Audiencia principal	Responsabilidad
README.md	Personas nuevas	Explicar producto, instalación y uso
AGENTS.md	Agentes de código	Reglas operativas, comandos, permisos, estilo y cierre
.github/copilot-instructions.md	GitHub Copilot	Instrucciones generales persistentes del repositorio
.github/instructions/*.instructions.md	Copilot por zona	Reglas aplicadas a paths concretos
.github/prompts/*.prompt.md	Equipo y asistentes	Prompts invocables para tareas repetidas
.github/orquestador/context/*	Equipo y agentes	Contexto estable: producto, arquitectura, glosario
.github/orquestador/sdd/*	Equipo y agentes	Specs, decisiones, tareas, trazabilidad, runbooks
.github/orquestador/pipelines/catalog.md	Maintainers	Inventario de workflows, permisos, riesgos y owners
.github/workflows/*	GitHub Actions	Automatización real: CI, reviewers, validaciones

La regla de oro: un archivo de contexto no debe fingir que ejecuta cosas. Un workflow sí ejecuta. Un catálogo documenta. Un prompt orienta. Un AGENTS.md gobierna el comportamiento de agentes. Mantener esa separación evita sistemas confusos.

Estructura recomendada

Una base razonable para un repo que quiere trabajar con IA, SDD y GitHub-first:

```
repo/  
  AGENTS.md  
  README.md  
  src/  
  tests/  
  docs/  
  .github/  
    copilot-instructions.md  
    instructions/  
      frontend.instructions.md  
      backend.instructions.md
```

```
tests.instructions.md
prompts/
  plan-feature.prompt.md
  write-spec.prompt.md
  review-pr.prompt.md
  generate-tests.prompt.md
  write-adr.prompt.md
  qa-harness.prompt.md
workflows/
  ci.yml
  pr-reviewer.yml
orquestador/
  README.md
  context/
    product.md
    architecture.md
    glossary.md
    constraints.md
  sdd/
    README.md
    requirements/
    specs/
    decisions/
    tasks/
    acceptance/
    traces/
    evaluations/
    runbooks/
    progress/
    examples/
  pipelines/
    catalog.md
  policies/
    permissions.md
    safety.md
```

No todos los proyectos necesitan todo desde el día uno. Pero conviene que la arquitectura mental exista. Un MVP puede empezar con `AGENTS.md`, `context/product.md`, `sdd/specs/`, `prompts/` y `workflows/ci.yml`. El resto se agrega cuando aparece repetición real.

Cómo escribir un buen `AGENTS.md`

`AGENTS.md` es el archivo que le dice a un agente cómo operar en el repositorio. No es una landing page. No es una visión de producto. No es un manual largo para humanos. Es una guía operacional.

Conviene afinar todavía más: `AGENTS.md` debe funcionar como **mapa**, no como enciclopedia. Su trabajo principal es decir qué leer, en qué orden y qué reglas no se pueden romper. Las reglas profundas pueden vivir en `docs/`, `specs/`, `progress/` o `.github/orquestador/`.

Debe responder:

- 1.Cuál es el stack.
2. Dónde está cada cosa importante.
3. Qué comandos se usan para instalar, probar, validar y correr.
4. Qué archivos o carpetas son delicados.
5. Qué estilo de cambios se espera.
6. Qué permisos o acciones están prohibidas.
7. Cómo cerrar una tarea.

Ejemplo base:

```
# AGENTS.md

## Project Snapshot

Este repo contiene una app Node.js + TypeScript con API en `src/server`,
frontend en `src/web` y tests en `tests`.

## Commands

- Instalar: `npm ci`
- Desarrollo: `npm run dev`
- Tests: `npm test`
- Typecheck: `npm run typecheck`
- Build: `npm run build`

## Working Rules

- Mantener cambios acotados al pedido.
- No modificar migraciones antiguas sin pedir permiso.
- No tocar secretos ni archivos `.env`.
- Preferir tests cerca del comportamiento cambiado.
- Si un comando falla, reportar el error exacto y el proximo paso.

## SDD

- Specs: `.github/orquestador/sdd/specs/`
- Decisiones: `.github/orquestador/sdd/decisions/`
- Tareas: `.github/orquestador/sdd/tasks/`
- Trazabilidad: `.github/orquestador/sdd/traces/`

## Completion

Antes de cerrar, informar:

- Archivos modificados.
- Pruebas ejecutadas.
- Pruebas no ejecutadas y por que.
- Riesgos residuales.
```

El error común es escribir un `AGENTS.md` demasiado filosofico. Un agente no necesita frases como "usar buenas practicas". Necesita comandos, rutas, límites y criterios.

Un `AGENTS.md` orientado a harness puede ser más explicito:

```
# AGENTS.md

## Antes de empezar

1. Ejecuta `./init.sh`.
2. Lee `progress/current.md`.
3. Lee `feature_list.json`.
4. Si la feature tiene `"sdd": true`, lee `docs/specs.md`.

## Mapa

| Ruta | Qué contiene | Cuándo leer |
| --- | --- | --- |
| `feature_list.json` | Estado de features | Siempre |
| `specs/<feature>/` | Requirements, design, tasks | Antes de implementar |
| `progress/current.md` | Estado vivo de sesión | Siempre |
| `docs/verification.md` | Cómo demostrar que funciona | Antes de cerrar |
| `CHECKPOINTS.md` | Criterios de estado final | Antes de declarar done |

## Reglas duras

- Una sola feature a la vez.
- No tocar código antes de spec aprobado.
- No declarar done sin `./init.sh` verde.
- No inventar contexto si existe en `docs/`.
```

La diferencia no es estética. Un `AGENTS.md` largo puede sonar completo y aun así fallar porque el agente no sabe que hacer primero. Un mapa corto con rutas correctas suele ser más efectivo.

Jerarquia de instrucciones

Las instrucciones tienen precedencia. Un pedido explicito del usuario tiene más peso que una regla general del repo. Un `AGENTS.md` más cercano a una subcarpeta puede especializar reglas del `AGENTS.md` raíz. Las instrucciones de sistema o plataforma tienen prioridad sobre todo lo demas.

La forma práctica de pensarlo:

```
Sistema / plataforma
> Usuario en la conversacion
  > AGENTS.md más cercano al archivo tocado
    > AGENTS.md raíz
      > Documentación auxiliar
```


Esto sirve especialmente en monorepos. Un root `AGENTS.md` puede decir como se trabaja globalmente, mientras `packages/mobile/AGENTS.md` define comandos y convenciones específicas de mobile.

`.github/copilot-instructions.md`

GitHub documenta las instrucciones personalizadas de repositorio para Copilot como un archivo `.github/copilot-instructions.md`. Su función es dar contexto persistente a Copilot cuando trabaja dentro del repositorio.

Este archivo debería ser más corto que `AGENTS.md`. No tiene que repetir todos los comandos. Puede enfocarse en estilo, arquitectura, preferencias de respuesta y validaciones esperadas.

Ejemplo:

```
# Copilot Instructions

Este proyecto usa TypeScript estricto. Evitar `any` salvo justificación.
Preferir funciones puras en la capa de dominio.
No crear dependencias nuevas sin explicar el motivo.
Cuando sugieras código, incluir pruebas relevantes.
Si falta contexto de negocio, preguntar o marcar supuesto.
```

Usalo para guiar la calidad general. No lo conviertas en un documento enorme. Si Copilot recibe demasiadas reglas generales, aumenta la probabilidad de conflictos o de que ignore parte del contenido.

`.github/instructions/*.instructions.md`

Las instrucciones por path permiten decir: "cuando trabajes en esta zona del repo, aplica estas reglas". GitHub documenta el patrón `NAME.instructions.md` dentro de `.github/instructions`, con frontmatter `applyTo`.

Ejemplo para backend:

```
---
applyTo: "src/server/**/*.ts,tests/server/**/*.ts"
---

# Backend Instructions

- Validar entradas en la frontera HTTP.
- Mantener reglas de negocio fuera de handlers.
- No acceder a la base de datos desde controllers.
- Agregar tests de casos borde para errores 4xx y 5xx.
```

Ejemplo para frontend:

```
---
applyTo: "src/web/**/*.tsx,src/web/**/*.css"
```

```
---

# Frontend Instructions

- Mantener componentes accesibles por teclado.
- Evitar texto que se desborde en mobile.
- Usar componentes existentes antes de crear nuevos.
- No introducir cambios visuales globales sin justificar.
```

La ventaja es precisión. En lugar de que una regla de frontend contamine backend, cada zona recibe lo que necesita.

`.github/prompts/*.prompt.md`

Los prompt files son comandos reutilizables. GitHub los documenta como archivos Markdown con extensión `.prompt.md`, normalmente dentro de `.github/prompts`. Están pensados para tareas que el equipo repite: planificar una feature, revisar un PR, generar tests, escribir una ADR, documentar una API o preparar un onboarding.

Alcance práctico: tratarlos como una capacidad de herramienta, no como estándar universal. En Copilot, la documentación actual los presenta como public preview y disponibles en IDEs concretos. Para que el repositorio siga siendo portable, el prompt debe poder leerse como Markdown aunque una herramienta no lo ejecute directamente.

Conviene tratarlos como comandos versionados. Si un prompt mejora, se revisa en PR. Si falla, se ajusta. Si deja de servir, se borra.

Ejemplo `plan-feature.prompt.md`:

```
---
agent: 'agent'
description: "Convertir una idea de producto en plan SDD por slices"
---

Usa el contexto de:
- [producto](../orquestador/context/product.md)
- [arquitectura](../orquestador/context/architecture.md)

Entrada del usuario:
${input:feature:Describe la feature}

Producir:
1. Problema y objetivo.
2. No objetivos.
3. Supuestos.
4. Requisitos funcionales.
5. Criterios de aceptación.
6. Slices de implementación.
7. Tests recomendados.
8. Riesgos y preguntas abiertas.
```

No inventes reglas de negocio. Marca lo desconocido.

Ejemplo `review-pr.prompt.md`:

```
---
agent: 'agent'
description: "Revisar un PR con foco en bugs, riesgos y pruebas"
---

Revisa los cambios del PR actual.

Prioridad:

1. Bugs o regresiones.
2. Riesgos de seguridad o permisos.
3. Falta de tests para comportamiento nuevo.
4. Inconsistencias con SDD o ADRs.

Salida:

- Findings con archivo y línea cuando sea posible.
- Preguntas abiertas.
- Resumen breve.

No hagas comentarios de estilo si no afectan mantenimiento o comportamiento.
```

Ejemplo `qa-harness.prompt.md`:

```
---
agent: 'agent'
description: "Diseñar un harness de evaluación para una capacidad de IA"
---

Capacidad a evaluar:
${input:capability:Describe la capacidad}

Diseña:

1. Fixtures de entrada.
2. Salidas esperadas o criterios.
3. Rúbrica de scoring.
4. Casos negativos.
5. Script CLI mínimo.
6. Cómo integrarlo en CI.
7. Riesgos de falsos positivos.
```

Estos archivos hacen que el conocimiento del equipo sea invocable. No reemplazan el juicio humano, pero reducen variabilidad.

.github/orquestador

La carpeta `.github/orquestador` funciona como casa del sistema de trabajo. El nombre no es magico. Lo importante es que tenga una responsabilidad clara: reunir contexto, SDD, políticas y catalogos que guian a humanos y agentes.

Una convención útil:

```
.github/orquestador/
  README.md           Indice del sistema operativo del repo
  context/            Contexto estable
  sdd/                Especificaciones y trazabilidad
  prompts/            Prompts internos si no se usan `.github/prompts`
  pipelines/          Catalogo de workflows y automatizaciones
  policias/           Permisos, seguridad, criterios de riesgo
```

Si el equipo usa mucho GitHub Copilot, conviene mantener `.github/prompts` para compatibilidad con herramientas y dejar `.github/orquestador` para contexto y SDD. Si se usa otro agente, se puede tener también `orquestador/prompts`. Lo importante es no duplicar sin necesidad.

GitHub-first, Claude-first o portable

Una organizacion GitHub-first funciona muy bien cuando el equipo vive en issues, PRs y CI. Pero no todos los equipos tienen la misma superficie real de ejecución.

Enfoque	Capa principal	Cuando conviene
GitHub-first	<code>.github/workflows</code> , <code>.github/prompts</code> , <code>.github/orquestador</code>	Cuando el trabajo vive en issues, PRs y CI
Claude-first	<code>.claude/agents</code> , <code>.claude/settings.json</code> , hooks locales	Cuando Claude Code es la herramienta diaria
Portable	<code>harness/</code> , <code>specs/</code> , <code>progress/</code> , <code>init.sh</code>	Cuando querés independencia de proveedor

Opción portable:

```
repo/
  AGENTS.md
  harness/
    feature_list.json
  init.sh
  CHECKPOINTS.md
  docs/
  specs/
```

```
progress/  
.github/  
workflows/  
prompts/  
orquestador/  
.claude/  
agents/  
settings.json
```

Opción GitHub-first:

```
repo/  
AGENTS.md  
.github/  
workflows/  
prompts/  
orquestador/  
feature_list.json  
CHECKPOINTS.md  
context/  
sdd/  
docs/  
specs/  
progress/  
pipelines/
```

Opción Claude-first:

```
repo/  
AGENTS.md  
CLAUDE.md  
feature_list.json  
init.sh  
specs/  
progress/  
docs/  
.claude/  
agents/  
leader.md  
spec_author.md  
implementer.md  
reviewer.md  
settings.json
```

La recomendación es elegir una superficie principal. Si GitHub es donde vive el equipo, usar GitHub-first. Si Claude Code es la herramienta diaria, agregar `.claude/` como capa operativa. Lo que no conviene es crear dos fuentes de verdad.

Árbol de decisión:

```
¿El trabajo se revisa y bloquea en PR?  
Sí -> GitHub-first.  
No -> ¿la herramienta diaria ejecuta hooks locales?  
Sí -> herramienta-first, con su carpeta operativa.  
No -> portable: AGENTS.md, specs/, progress/ e init.sh.
```

Fuente de verdad: elegir una carpeta para specs y una para estado vivo. Las demás superficies pueden apuntar a esa fuente, pero no duplicarla. Migrar significa mover la fuente de verdad y dejar enlaces o notas de transición, no mantener dos copias activas.

SDD dentro del repo

SDD significa trabajar desde especificaciones, no desde impulsos. En un repositorio con IA, SDD cumple una función crítica: evita que el agente implemente una interpretación creativa del pedido.

Una spec mínima debería incluir:

1. Problema.
2. Objetivo.
3. No objetivos.
4. Requisitos.
5. Criterios de aceptación.
6. Casos borde.
7. Impacto técnico.
8. Plan por slices.
9. Pruebas.
10. Trazabilidad con issue, PR y decisiones.

Ejemplo de ruta:

```
.github/orquestador/sdd/specs/reasignar-reclamo.md
```

Ejemplo de encabezado:

```
# Reasignar reclamo a otro equipo  
  
Estado: propuesto  
Owner: Producto / Backend  
Issue: #123  
PRs: pendiente  
  
## Problema  
  
Los operadores no pueden mover un reclamo cuando fue derivado al equipo incorrecto.
```

Criterios de aceptación

- Un operador autorizado puede reasignar el reclamo.
- La reasignación queda auditada.
- El equipo anterior y el nuevo quedan visibles en el historial.
- Si el reclamo está cerrado, no puede reasignarse.

Cuando el agente implemente, debe poder leer esa spec y saber que significa "terminado".

Workflows como gate remoto

Si el repositorio usa GitHub como superficie principal, conviene que `.github/workflows` sea el gate remoto principal. Eso no invalida `init.sh` ni hooks locales: esos viven antes del PR y ayudan a fallar rápido. La regla es elegir una fuente canonica por responsabilidad.

Responsabilidad	Fuente canonica recomendada
Validación local antes de cerrar una tarea	<code>init.sh</code>
Guardrails locales de una herramienta	hooks de la herramienta
Gate remoto de PR/CI	<code>.github/workflows/*</code>
Catalogo y gobierno de automatizaciones	<code>.github/orquestador/pipelines/catalog.md</code>

Esta separación evita dos problemas: automatizaciones duplicadas y dudas sobre dónde ocurre realmente la ejecución.

Patrón conservador para empezar:

```
name: Safe PR Reviewer

on:
  pull_request:
    types: [opened, synchronize, reopened]

permissions:
  contents: read
  pull-requests: read
  issues: write

concurrency:
  group: pr-reviewer-${{ github.event.pull_request.number }}
  cancel-in-progress: true

jobs:
  review:
    runs-on: ubuntu-latest
```

```

steps:
  - name: Comment with checklist
    uses: actions/github-script@v7
    with:
      script: |
        const body = [
          "Revisión automática inicial:",
          "- Verificar tests.",
          "- Confirmar criterios SDD.",
          "- Revisar permisos y secretos.",
          "- Mantener cambios acotados."
        ].join("\n");
        await github.rest.issues.createComment({
          owner: context.repo.owner,
          repo: context.repo.repo,
          issue_number: context.payload.pull_request.number,
          body
        });

```

Este workflow no hace checkout, no ejecuta código del PR y comenta de forma conservadora. Debe leerse como un **notificador no bloqueante**, no como enforcement técnico. Para enforcement real, agregar un workflow separado de CI que ejecute tests, lint, typecheck o validaciones del harness.

Catalogo de pipelines

El catálogo no ejecuta. Documenta. Debería responder:

1. Qué workflow existe.
2. Qué evento lo dispara.
3. Qué permisos usa.
4. Qué riesgos tiene.
5. Quién lo mantiene.
6. Qué output produce.
7. Qué no está autorizado a hacer.

Ejemplo:

```

# Pipeline Catalog

## safe-pr-reviewer

- Archivo: `.github/workflows/pr-reviewer.yml`
- Evento: `pull_request`
- Permisos: `contents: read`, `pull-requests: read`, `issues: write`
- Ejecuta código del PR: no
- Output: comentario con checklist
- Owner: Platform

```


- Riesgo principal: ruido en PRs si las reglas no se mantienen
- Estado: activo

Cuando alguien pregunte "qué automatizaciones tiene este repo", el catálogo da la respuesta. Cuando alguien pregunte "qué bloquea un PR", la respuesta debe estar en `.github/workflows`. Cuando pregunte "qué valida localmente el agente", la respuesta debe estar en `init.sh` o en los hooks de la herramienta.

Cómo se conectan las piezas

Un flujo completo podría ser:

1. El humano crea un issue.
2. Ejecuta `plan-feature.prompt.md` para convertir la idea en spec.
3. Guarda la spec en `.github/orquestador/sdd/specs/`.
4. El agente lee `AGENTS.md`, contexto y spec.
5. El agente implementa un slice acotado.
6. Corre tests indicados por `AGENTS.md`.
7. Abre PR.
8. `pr-reviewer.yml` comenta checklist como notificador no bloqueante.
9. Otro agente o humano usa `review-pr.prompt.md`.
10. Se actualiza trazabilidad en SDD.

Walkthrough compacto:

```
Issue: "agregar notes recent"
Prompt: plan-feature.prompt.md crea spec y tasks.
SDD: requirements/recent-notes.md define R1, R2, R3.
Subagentes: explorer revisa store; implementer toca solo CLI y tests.
Harness: init.sh corre unit, lint y validate-sdd.
Review: reviewer valida R1/R2/R3 contra tests.
Cierre: progress/history.md registra evidencia y PR.
```

Lo valioso es que cada paso deja rastros. El equipo no depende de recordar que se habló en un chat.

Errores frecuentes

Error	Consecuencia	Correccion
<code>AGENTS.md</code> enorme	El agente ignora partes	Mantenerlo operativo y linkear docs largas
Repetir reglas en cinco archivos	Instrucciones conflictivas	Definir owner por capa
Prompts sin versionar	Cada persona usa variantes	Guardarlos en <code>.github/prompts</code>
Workflows con permisos amplios	Riesgo innecesario	Permisos minimos por workflow
Catalogo que promete ejecutar	Confusion operativa	Catalogo documenta, workflows ejecutan
Specs sin criterios de aceptación	Implementaciones ambiguas	Cerrar cada spec con pruebas y ejemplos

Checklist de bootstrap

Para dejar un repo listo:

1. Crear `AGENTS.md` con comandos reales.
2. Crear `.github/copilot-instructions.md` breve.
3. Crear `.github/instructions/` solo si hay reglas por path.
4. Crear `.github/prompts/` con 3 a 5 prompts útiles.
5. Crear `.github/orquestador/context/product.md`.
6. Crear `.github/orquestador/context/architecture.md`.
7. Crear `.github/orquestador/sdd/README.md`.
8. Crear `specs/`, `decisions/`, `tasks/`, `acceptance/`, `traces/`, `evaluations/`, `runbooks/` y `progress/` si el flujo lo necesita.
9. Crear `.github/orquestador/pipelines/catalog.md`.
10. Verificar que `.github/workflows` sea el gate remoto principal y que `init.sh` cubra la validación local.
11. Hacer un PR de prueba para confirmar que las instrucciones son entendibles.

Fuentes verificadas

GitHub documenta las instrucciones de repositorio para Copilot, incluyendo `.github/copilot-instructions.md`, `.github/instructions/*.instructions.md` y el uso de `AGENTS.md` para instrucciones de agentes:

- [GitHub Docs - Adding repository custom instructions for GitHub Copilot](#)
- [GitHub Docs - Prompt files](#)
- [GitHub Docs - Your first prompt file](#)

- [openai/agents.md](#)

Antes de convertir una convención de herramienta en estándar interno, conviene contrastarla con la documentación oficial vigente y con la forma real en que trabaja el equipo.

Cierre

La meta no es tener una carpeta `.github` impresionante. La meta es que cada agente que entra al repo pueda trabajar con menos adivinanza, más verificación y mejor memoria. `AGENTS.md` da las reglas de operación. SDD da el contrato de producto y técnica. Los prompt files convierten tareas repetidas en comandos. Los workflows ejecutan validaciones. El catálogo explica el sistema.

Cuando esas piezas están alineadas, la IA deja de ser un chat suelto y empieza a ser infraestructura de trabajo.

VOLUMEN 05

Prompt Engineering y Harness Engineering

De prompts sueltos a sistemas versionados, evaluables y productivos

Al terminar este volumen podés transformar prompts sueltos en harnesses evaluables, trazables y productivos.

La diferencia entre un experimento útil y un sistema confiable no está solo en escribir un buen prompt. Un prompt aislado puede resolver una tarea puntual, pero un producto serio necesita algo más: versiones, casos de prueba, criterios de evaluación, observabilidad y reglas de seguridad. Ese conjunto es lo que convierte una idea en una capacidad operable.

Este volumen parte de una idea simple: un prompt no debería vivir como una cadena suelta pegada en una app, en un notebook o en un comentario. Si una instrucción importa para el negocio, tiene que poder auditarse, compararse, probarse y desplegarse con disciplina. Ahí entra el harness engineering: el diseño del entorno que ejecuta, mide y controla ese prompt.

La práctica cambia el foco. Ya no se trata de preguntarse “¿qué le digo al modelo?”, sino “¿cómo hago para que esta tarea se ejecute siempre con el mismo criterio, se pueda verificar y no se rompa cuando el sistema crece?”. Esa es la transición de prompt engineering artesanal a ingeniería de prompts productiva.

Qué cambia cuando el prompt entra en producción

En una demo, un prompt puede ser el texto exacto que hoy te dio buen resultado. En producción, ese mismo texto deja de ser suficiente porque aparecen condiciones que antes no importaban:

1. El modelo cambia.
2. La temperatura cambia.
3. Cambia el contexto del usuario.
4. Cambian los datos de entrada.
5. Cambia el equipo que mantiene el sistema.
6. Aparecen requisitos legales, de seguridad o de marca.

Cuando eso pasa, el problema no es solo “calidad de respuesta”. El problema es control. Necesitás saber qué instrucción se usó, con qué versión, sobre qué inputs, con qué herramientas, y si el comportamiento sigue dentro de lo esperado.

Por eso, un sistema de prompts productivo suele tener estas piezas:

- instrucciones persistentes, que describen la identidad y el comportamiento estable;
- prompts de tarea, que resuelven una solicitud concreta;
- fixtures, que representan casos de entrada bien definidos;
- golden tests, que fijan salidas esperadas o propiedades observables;
- evals, que miden calidad con una rúbrica;
- regression tests, que detectan degradaciones;
- reglas de permisos y seguridad;
- logging y observabilidad para revisar resultados en contexto.

No son capas decorativas. Son la infraestructura mínima para confiar en un asistente, un clasificador, un redactor, un router o un agente.

Anatomía de un prompt bueno

Un prompt bueno no es largo ni corto por definición. Es útil cuando reduce ambigüedad, ordena prioridades y deja claro qué hacer cuando faltan datos. En la práctica, los mejores prompts tienen una estructura parecida.

1. Objetivo explícito

El modelo tiene que saber qué problema resuelve. No alcanza con “ayudá al usuario”. Conviene especificar el objetivo en términos operativos.

Ejemplo:

```
Tu tarea es transformar pedidos informales en tickets de trabajo claros, accionables y completos.
```

2. Contexto de operación

El prompt tiene que decir en qué entorno se usa y qué límites tiene. No es lo mismo redactar para soporte, para un agente de ventas o para un clasificador interno.

Ejemplo:

```
Este asistente se usa en un equipo de operaciones. Prioriza claridad, trazabilidad y lenguaje profesional.
```

3. Criterios de calidad

Si no definís calidad, el modelo la inventa. Un buen prompt marca lo que se valora: precisión, concisión, cobertura, tono, seguridad, formato.

Ejemplo:

```
La respuesta debe ser concreta, no inventar datos, y separar hechos observables de supuestos.
```

4. Restricciones

Las restricciones evitan respuestas cómodas pero inútiles. Acá entran formato, longitud, herramientas disponibles, idiomas, exclusiones y normas de seguridad.

Ejemplo:

```
No uses tablas si la información es incierta. No asumas permisos. No ejecutes acciones sin confirmación.
```

5. Política de incertidumbre

Un prompt maduro indica qué hacer cuando falta contexto. Eso reduce alucinaciones y respuestas temblorosas.

Ejemplo:

```
Si falta un dato crítico, hacé una sola pregunta de aclaración. Si el dato no es crítico, seguí con una suposición explícita.
```

6. Formato de salida

El output debe ser fácil de consumir por humanos o por otra capa del sistema. Si necesitás JSON, marcá el esquema. Si necesitás viñetas, decilo. Si necesitás una plantilla, definila.

Ejemplo:

```
Devolvé siempre:  
1. Resumen  
2. Riesgos  
3. Próximos pasos
```

7. Ejemplos

Los ejemplos no son adorno. Son anclas semánticas. Ayudan a fijar tono, nivel de detalle y decisiones límite. En especial sirven cuando la tarea tiene ambigüedad de formato o de criterio.

Un prompt bueno suele mezclar instrucciones con 1 o 2 ejemplos compactos. No hace falta sobrecargarlo: demasiados ejemplos también introducen ruido.

Instrucciones persistentes vs prompts de tarea

Una confusión común es mezclar todo en una sola instrucción. Eso vuelve el sistema frágil. La división útil es esta:

- instrucciones persistentes: lo que no debería cambiar entre tareas;
- prompts de tarea: lo que cambia para cada pedido;
- contexto dinámico: datos del usuario, archivos, estado de la sesión, herramientas, memoria temporal.

Instrucciones persistentes

Son la capa de identidad y política. Definen el rol, el tono, la prioridad de criterios, los límites de seguridad, el idioma y la conducta esperada.

Ejemplo:

```
Sos un asistente de operaciones. Respondés en español rioplatense neutro, con estilo claro y serio. Priorizás precisión, seguridad y pasos accionables.
```

Prompts de tarea

Son instrucciones puntuales para una ejecución concreta. Deben ser breves, específicas y enfocadas en el resultado.

Ejemplo:

```
Con este texto, redactá un resumen ejecutivo de máximo 180 palabras y terminá con 3 riesgos concretos.
```

Regla práctica

Si una instrucción se repite en todos los casos, probablemente va en la capa persistente. Si cambia por cada ejecución, va en la capa de tarea. Si vive en el input del usuario, no la mezcles con política. Separar estas cosas reduce errores y hace que el sistema sea mantenible.

Fixtures: casos concretos para probar comportamiento

Un fixture es un caso de entrada controlado que representa una situación realista. En prompt engineering, los fixtures son esenciales porque permiten verificar si el sistema responde bien ante escenarios típicos, raros o peligrosos.

No alcanza con un solo ejemplo feliz. Un harness serio necesita fixtures que cubran:

- entradas limpias;
- entradas incompletas;
- entradas contradictorias;
- entradas con ruido;
- pedidos ambiguos;
- intentos de forzar una salida insegura;
- casos límite de formato.

Ejemplo de fixture

```
{
  "id": "ticket-001",
  "input": "Necesito que alguien revise la factura del mes pasado porque creo que vino mal.",
  "expected_traits": [
    "pide aclaracion si falta identificador",
    "no inventa datos",
    "mantiene tono profesional",
    "propone proximo paso"
  ]
}
```

Qué hace bueno a un fixture

Un buen fixture no intenta “ganarle” al modelo con trucos. Describe una situación útil. Tiene que ser:

- estable;
- reproducible;
- legible;
- representativo;
- fácil de ampliar.

Si el fixture cambia todo el tiempo, no sirve para comparar versiones. Si es demasiado artificial, no refleja el uso real. La calidad está en el balance.

Golden tests: fijar comportamiento esperado

Los golden tests son pruebas contra una salida esperada o contra propiedades muy concretas de la salida. Sirven para detectar regresiones cuando cambian el prompt, el modelo o la cadena de herramientas.

Hay dos formas comunes:

Golden exacto

Sirve cuando la salida debe ser muy estable, por ejemplo un formato estructurado.

Ejemplo:

```
{
  "id": "routing-01",
  "expected": {
    "category": "facturacion",
    "confidence": "alta"
  }
}
```

Golden por propiedades

Sirve cuando no querés congelar el texto completo, pero sí el comportamiento.

Ejemplo:

- la respuesta no debe inventar un dato;
- debe incluir una advertencia;
- debe devolver exactamente 3 pasos;
- debe usar el idioma esperado;
- no debe mencionar herramientas no autorizadas.

Este segundo formato es más flexible y suele ser más útil para sistemas con lenguaje natural. El oro no siempre es el string exacto; muchas veces es el cumplimiento de reglas observables.

Harness CLI: correr, comparar, repetir

El harness es el entorno que ejecuta prompts con fixtures, registra resultados y los compara contra una referencia. Una CLI bien pensada hace que todo eso sea repetible desde terminal y CI.

Una estructura razonable separa:

- definición del prompt;
- definición de fixtures;
- configuración del modelo;
- ejecución del lote;
- evaluación;

- reporte;
- exportación de resultados.

Estructura posible

```
prompt-harness/  
  prompts/  
    system.md  
    task.md  
  fixtures/  
    inbox.jsonl  
    safety.jsonl  
    formatting.jsonl  
  evals/  
    rubric.md  
    scoring.ts  
  runs/  
    run-001.jsonl  
  reports/  
    latest.md  
  src/  
    cli.ts  
    harness.ts  
    loader.ts  
    evaluator.ts
```

Comandos ejemplo

```
node src/cli.js run --prompt prompts/system.md --fixtures fixtures/inbox.jsonl  
node src/cli.js eval --run runs/latest.jsonl --rubric evals/rubric.md  
node src/cli.js compare --baseline runs/baseline.jsonl --candidate runs/latest.jsonl  
node src/cli.js report --input runs/latest.jsonl --output reports/latest.md
```

Qué debería hacer la CLI

Una CLI útil no solo llama al modelo. También:

- valida que los archivos existan;
- normaliza entradas;
- registra versión de prompt y modelo;
- guarda timestamps;
- serializa respuestas crudas;
- calcula métricas;
- emite un resumen claro para CI.

Si el harness solo imprime texto bonito, es demo. Si también deja trazabilidad y comparación, ya empieza a ser infraestructura.

init.sh como contrato de salud

En un harness de desarrollo, el script más importante no siempre es el más sofisticado. Muchas veces es un único comando:

```
./init.sh
```

Ese script debería responder una pregunta simple: "este repo está en condiciones de seguir?". Para eso puede:

1. Validar que las dependencias existen.
2. Ejecutar tests.
3. Correr linters o typechecks.
4. Validar estructura de specs.
5. Confirmar que no haya más de una feature `in_progress`.
6. Fallar si falta trazabilidad entre requirements y tests.

Regla operacional:

Si `./init.sh` está rojo, la feature no puede estar `done`.

El objetivo no es reemplazar CI. El objetivo es darle al agente una puerta local clara. Si cada cierre termina con "ejecute `./init.sh`, resultado verde", el equipo puede revisar con menos ambigüedad.

Contrato mínimo recomendado:

```
0 = todo verde
1 = falla de validación o tests
2 = falta configuración local
3 = estructura SDD invalida
4 = permisos insuficientes o acción bloqueada
```

También conviene que el script sea idempotente: correrlo dos veces no debería cambiar el repo ni depender de estado invisible. Si tarda mucho, separa chequeos rapidos de suites pesadas y documenta ambos comandos. El agente necesita una puerta local confiable, no un ritual impredecible.

Ejemplo mínimo:

```
#!/usr/bin/env sh
set -eu

command -v node >/dev/null 2>&1 || exit 2
npm test || exit 1
npm run lint || exit 1
node scripts/validate-sdd.mjs || exit 3
node scripts/check-permissions.mjs || exit 4
```

Si el equipo trabaja principalmente en Windows, puede existir un `init.ps1` o un `node scripts/init.mjs` equivalente. Lo importante es que haya un comando de cierre documentado, repetible y con códigos de salida claros.

Trazabilidad requirement-test

Un harness serio no se conforma con "hay tests". Necesita saber que test cubre que requirement.

Ejemplo:

```
R1 -> test_recent_default_limit
R2 -> test_recent_invalid_limit
R3 -> test_recent_empty_store
```

La trazabilidad puede vivir en:

- `specs/<feature>/requirements.md` ;
- `specs/<feature>/tasks.md` ;
- `progress/review_<feature>.md` ;
- una tabla generada por el harness.

El reviewer debería rechazar si:

1. Un `R<n>` no tiene test.
2. Un test nuevo no se vincula a ningún requirement.
3. Una task dice cubrir un requirement pero el test no lo demuestra.
4. El código implementa comportamiento que no aparece en el spec.

Esto evita una trampa común: agregar tests que ejercitan código, pero no verifican el contrato real.

Una tabla de traza útil tiene esta forma:

Requirement	Test	Evidencia	Estado
R1	<code>test_recent_default_limit</code>	<code>tests/test_recent.py</code>	cubierto
R2	<code>test_recent_invalid_limit</code>	<code>tests/test_recent.py</code>	cubierto
R3	<code>test_recent_empty_store</code>	<code>tests/test_recent.py</code>	cubierto

La regla de cierre es simple: ninguna feature queda terminada si su tabla de traza tiene huecos.

Hooks y workflows como guardrails

Si una regla es crítica, no debería depender solo de que el agente recuerde obedecerla. Conviene automatizarla.

En una superficie Claude-first, los hooks pueden ejecutar un chequeo liviano después de editar y una validación completa antes de cerrar:

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          {
            "type": "command",
            "command": "npm run check:quick"
          }
        ]
      }
    ],
    "Stop": [
      {
        "hooks": [
          {
            "type": "command",
            "command": "npm run check:full"
          }
        ]
      }
    ]
  }
}
```

En una superficie GitHub-first, el equivalente son workflows:

```
.claude/settings.json  -> hooks locales para Claude Code
.github/workflows/*    -> validación remota en PR/CI
```

El principio es el mismo: convertir reglas importantes en verificación ejecutable.

Los comandos son ejemplos. En un repo Python, Go, Rust o Windows-first, conviene usar el comando natural del proyecto. La regla importante no es el nombre del runtime, sino separar chequeos rápidos de cierre completo.

El detalle importante es como fallan. Un hook que imprime una advertencia pero permite continuar no bloquea nada. Si una regla es obligatoria, el comando debe devolver código de salida distinto de cero y dejar un mensaje corto que explique que corregir. Si la regla es solo informativa, debe decirlo explícitamente para no confundirse con un gate real.

Checklist Harness-SDD

Antes de cerrar una feature asistida por IA:

- [] Hay una sola feature activa.
- [] `feature\_list.json` refleja el estado real.
- [] Si `sdd: true`, existen `requirements.md`, `design.md`, `tasks.md`.
- [] El humano aprobo el spec antes de tocar código.
- [] Cada requirement tiene id `R<n>`.
- [] Cada `R<n>` tiene al menos un test.
- [] Cada task referencia uno o más requirements.
- [] El implementer marco tasks completadas.
- [] El reviewer valido trazabilidad.
- [] `./init.sh` termina verde.
- [] `progress/current.md` se cerró o se marco como blocked.
- [] `progress/history.md` recibió el resumen final.

Esta lista es deliberadamente estricta. La IA moderna no se controla con prompts cada vez más largos. Se controla con harnesses: archivos, estados, specs, permisos, tests y revisiones que hacen que el agente no pueda saltarse el proceso sin dejar evidencia.

Evals y rúbricas: medir más allá de “me gusta”

La evaluación de prompts no puede depender solo de intuición. Necesitás criterios repetibles. Ahí entra la rúbrica: una definición explícita de qué significa “bueno”.

Convención de nombres: en SDD, `evaluations/` suele guardar notas, conclusiones y decisiones de evaluación. En un harness, `evals/` suele contener piezas ejecutables o semi-ejecutables: rúbricas, scoring, fixtures y scripts. Pueden conectarse, pero no cumplen la misma función. `evals/` mide; `evaluations/` deja la postura y la decisión.

Rúbrica simple

Una rúbrica puede puntuar dimensiones como:

- fidelidad al contexto;
- completitud;
- exactitud;
- formato;
- tono;
- seguridad;
- utilidad accionable.

Ejemplo:

```
Puntaje 0-2 por dimensión:  
0 = falla grave
```

```
1 = parcial
2 = cumple
```

Umbrales posibles:

```
promedio >= 1.7 y ninguna dimension crítica en 0 -> aprobable
promedio entre 1.2 y 1.69 -> requiere revisión
promedio < 1.2 o seguridad = 0 -> rechazado
```

El umbral no tiene que ser universal. Tiene que estar escrito antes de evaluar, porque cambiar la regla después de ver el resultado convierte la evaluación en opinion.

Ejemplo `evals/rubric.md`:

```
# Rúbrica de respuesta operativa

## Dimensiones

- Fidelidad al contexto: no inventa datos ni contradice fuentes.
- Completitud: cubre todos los puntos pedidos.
- Accionabilidad: deja próximos pasos verificables.
- Seguridad: respeta permisos y bloqueos.
- Formato: entrega la estructura solicitada.

## Regla de decisión

- Aprobable: promedio >= 1.7 y seguridad > 0.
- Revisión: promedio entre 1.2 y 1.69.
- Rechazado: promedio < 1.2 o seguridad = 0.
```

Ejemplo de evaluación

Dimensión	Criterio
Fidelidad	No inventa datos ni contradice el input
Formato	Respetar la estructura pedida
Tono	Mantiene lenguaje profesional
Accionabilidad	Da próximos pasos útiles
Seguridad	No ejecuta ni recomienda acciones prohibidas

Evaluación automática y humana

Conviene combinar ambos enfoques.

- automática: para reglas objetivas, formato, longitud, presencia de campos, patrones prohibidos;

- humana: para matices de calidad, claridad, persuasión, utilidad y alineación.

En sistemas reales, el error más común es usar métricas automáticas pobres como si resolvieran todo. Sirven, pero no reemplazan la lectura crítica. La buena práctica es usar automatización para filtrar y rúbrica humana para decidir.

Regression tests: que no se rompa lo que ya andaba

Un regression test compara comportamiento entre versiones. El objetivo no es congelar el sistema para siempre, sino detectar cambios no deseados.

En un flujo maduro, cada cambio de prompt o de modelo debería responder:

1. qué mejoró;
2. qué empeoró;
3. qué casos nuevos aparecieron;
4. qué se acepta como trade-off;
5. qué necesita rollback.

Casos de regresión típicos

- el prompt nuevo empieza a ser más verboso de lo necesario;
- desaparece una advertencia de seguridad;
- el modelo deja de pedir aclaraciones;
- cambia el idioma;
- el formato JSON rompe un parser;
- una herramienta se invoca cuando no debería.

Estrategia práctica

Mantén un set pequeño de fixtures críticos y un set más amplio de fixtures exploratorios. Los críticos protegen lo esencial. Los exploratorios te muestran cómo se comporta el sistema fuera del camino principal.

Si una regression test falla, no basta con “arreglar el output”. Hay que entender si el problema está en:

- el prompt;
- el modelo;
- el postprocesado;
- la configuración;
- la política;
- el set de datos.

Seguridad y permisos

Cuando un modelo produce acciones, no alcanza con que responda bien. También tiene que actuar dentro de límites claros. En sistemas con herramientas, la seguridad forma parte del diseño del prompt y del harness.

Principios básicos

- mínimo privilegio;
- confirmación para acciones sensibles;
- separación entre sugerir y ejecutar;
- validación de input y output;
- logging de decisiones;
- bloqueo explícito de acciones peligrosas.

Ejemplo de regla

Si una acción modifica datos, cobra dinero, borra información o envía mensajes externos, pedir confirmación humana antes de ejecutarla.

Prompt seguro

El prompt no debe dar por sentados permisos ni credenciales. Tampoco debería alentar al modelo a “resolver por su cuenta” cosas que requieren validación externa.

Ejemplo:

No asumas que tenés acceso a sistemas externos. Si hace falta una acción con impacto, describila y pedí confirmación.

Harness seguro

El harness debe poder simular permisos y probar límites:

- sin acceso a internet;
- con herramientas deshabilitadas;
- con credenciales falsas;
- con modo solo lectura;
- con confirmación requerida.

Eso permite verificar que el sistema no solo funciona cuando todo está habilitado, sino también cuando opera bajo restricciones.

Observabilidad: ver qué pasó de verdad

La observabilidad es lo que permite depurar y aprender del sistema una vez que salió del laboratorio. Si un prompt falla en producción, necesitás reconstruir el contexto.

Qué conviene registrar

- versión del prompt;
- versión del modelo;

- fecha y hora;
- entrada resumida;
- salida cruda;
- herramientas usadas;
- latencia;
- costo estimado;
- errores;
- decisión de policy;
- identificador del fixture o caso.

Ejemplo de log

```
{
  "run_id": "run-001",
  "prompt_version": "1.4.2",
  "model": "provider-model-id",
  "fixture_id": "safety-03",
  "latency_ms": 1840,
  "tools_used": ["search"],
  "outcome": "needs_review"
}
```

Higiene de logs: no guardes secretos, tokens, datos personales innecesarios ni prompts internos que no puedan circular. Para auditoría suele alcanzar con identificadores, hashes, resumen del input, salida relevante y punteros a artefactos controlados.

Qué mirar primero

Cuando algo falla, conviene revisar:

1. el input original;
2. el prompt real aplicado;
3. la configuración del modelo;
4. los tools disponibles;
5. la salida cruda;
6. el postprocesado;
7. el criterio de evaluación.

Sin observabilidad, cada bug parece magia. Con observabilidad, se vuelve una secuencia de decisiones auditables.

Un ejemplo mínimo de harness

Para aterrizar la idea, imaginá un sistema que clasifica solicitudes internas y responde con un plan breve. La arquitectura mínima puede verse así:

1. cargar instrucciones persistentes
2. cargar prompt de tarea
3. cargar fixture
4. construir mensaje final
5. ejecutar modelo
6. validar formato
7. puntuar con rúbrica
8. guardar resultado
9. comparar contra baseline
10. reportar

Pseudocódigo

```
const system = loadFile("prompts/system.md")
const task = loadFile("prompts/task.md")
const fixtures = loadJsonl("fixtures/inbox.jsonl")

for (const fixture of fixtures) {
  const input = buildInput(system, task, fixture)
  const output = await model.run(input)
  const score = evaluate(output, fixture.expected_traits)
  saveRun({ fixtureId: fixture.id, output, score })
}
```

Lo importante del ejemplo

No hace falta sofisticación para empezar. Lo esencial es que el flujo sea:

- explícito;
- reproducible;
- versionado;
- evaluable;
- comparable.

Una vez que eso existe, se puede escalar. Antes de eso, todo es difícil de mantener.

Criterios para iterar prompts sin romper el sistema

Cuando mejorás un prompt, no conviene editarlo a ciegas. Es mejor seguir un ciclo corto y disciplinado.

Checklist de iteración

- definir el problema exacto;
- elegir un fixture representativo;
- escribir la hipótesis de mejora;
- cambiar una sola cosa por vez;

- correr el harness;
- comparar contra baseline;
- revisar fallos nuevos;
- aceptar el cambio o revertirlo;
- documentar la decisión.

Reglas prácticas

1. No mezcles mejoras de formato con cambios de política en la misma iteración.
2. No uses un solo ejemplo feliz para justificar un cambio.
3. No declares victoria sin mirar regresiones.
4. No guardes un prompt nuevo sin su versión y su motivo.
5. No metas lógica de negocio crítica solo dentro del prompt si puede vivir mejor en código.

Cuándo conviene mover lógica fuera del prompt

Un prompt no debería cargar con todo. Si una regla es estricta, verificable y central al negocio, muchas veces conviene codificarla fuera del modelo.

Ejemplos:

- validación de esquema;
- reglas de permisos;
- enrutamiento determinístico;
- filtrado de datos sensibles;
- normalización de formatos;
- cálculo de métricas.

El prompt queda para lo que hace mejor: interpretar, priorizar, redactar, sintetizar y decidir con contexto. El código queda para lo que hace mejor: validar, controlar y ejecutar de forma determinista.

Señales de madurez

Sabés que pasaste de "prompt suelto" a "sistema" cuando podés responder estas preguntas sin improvisar:

- ¿qué versión del prompt está en producción?
- ¿con qué fixtures se valida?
- ¿qué criterios define la rúbrica?
- ¿qué cambió respecto del baseline?
- ¿qué acciones están permitidas?
- ¿qué quedó registrado en los logs?
- ¿qué regresiones se toleran y cuáles no?

Si esas respuestas están dispersas, el sistema todavía depende demasiado de memoria humana.

Resumen operativo

Prompt engineering no es solo redactar mejor. Es diseñar una interfaz de instrucción que se pueda sostener en el tiempo. Harness engineering es el soporte que hace posible esa sostenibilidad: carga fixtures, ejecuta versiones, evalúa resultados, registra evidencia y protege el sistema de cambios accidentales.

Frontera práctica:

Decisión	Vive mejor en	Ejemplo
Tono, rol, formato y criterios blandos	Prompt	"Responder en español claro con hallazgos priorizados"
Reglas deterministas, permisos y validaciones duras	Código	"rechazar <code>limit > 50</code> "
Casos de prueba, comparación y evidencia	Harness	fixtures, scoring, reportes y baseline
Decisiones de producto o riesgo	Evaluación humana	aceptar trade-off, pedir rollback, cambiar alcance

Si algo debe cumplirse siempre, no debería depender solo de una frase en el prompt. Debe estar validado por código, test, harness o revisión humana explícita.

La disciplina práctica queda en una fórmula simple:

- separar instrucciones persistentes de prompts de tarea;
- usar fixtures reales y representativos;
- fijar expectativas con golden tests y rúbricas;
- correr regression tests antes de publicar cambios;
- controlar permisos y acciones sensibles;
- observar lo que pasa en producción;
- versionar todo lo que importe.

Cuando eso existe, el prompt deja de ser una apuesta. Se vuelve una pieza de ingeniería.

Checklist final

- ☐ Definir la instrucción persistente del sistema.
- ☐ Separar el prompt de tarea del contexto dinámico.
- ☐ Crear fixtures para casos felices, ambiguos y peligrosos.
- ☐ Escribir al menos un golden test por comportamiento crítico.

- ☐ Diseñar una rúbrica simple y repetible.
- ☐ Implementar un harness CLI con `run`, `eval` y `report`.
- ☐ Guardar versionado de prompts, modelo y configuración.
- ☐ Registrar logs y latencia por ejecución.
- ☐ Probar permisos, fallbacks y casos sin herramientas.
- ☐ Comparar cada cambio contra un baseline.
- ☐ Documentar decisiones de cambio y rollback.

Ese conjunto alcanza para pasar de una idea prometedora a una capacidad productiva, auditable y mantenible.

Ejemplos y Casos de Uso

Scaffolding completo para aplicar La Biblia Moderna en un repositorio real

Al terminar este volumen tenés un scaffolding completo para aplicar toda la Biblia Moderna en un repositorio real.

Este volumen cierra la explicación con un caso completo. La fecha de corte de este ejemplo es **15/05/2026**. La idea no es mostrar una maqueta decorativa, sino un scaffolding funcional que una persona pueda copiar, adaptar y usar como base para un repositorio con IA, SDD, subagentes, prompts reutilizables, validación local, CI y harness.

El sistema elegido es pequeño a propósito: una CLI de notas llamada `agentic-notes`. Permite agregar notas, listar notas recientes y buscar texto. El dominio es simple, pero la estructura de trabajo es robusta. Eso permite ver con claridad qué vive en cada archivo y cómo se conectan las piezas.

Estructura final

```
agentic-notes/  
  AGENTS.md  
  README.md  
  package.json  
  init.sh  
  src/  
    cli.mjs  
    store.mjs  
  test/  
    notes.test.mjs  
  scripts/  
    validate-sdd.mjs  
  harness/  
    fixtures/  
      recent.jsonl  
    evals/  
      rubric.md  
      run-eval.mjs  
  .github/  
    copilot-instructions.md  
  instructions/  
    tests.instructions.md  
  prompts/  
    plan-feature.prompt.md  
    review-pr.prompt.md  
  workflows/  
    ci.yml  
  orquestador/  
    context/  
      product.md  
      architecture.md  
    sdd/  
      requirements/  
        recent-notes.md  
      specs/  
        recent-notes.md  
      tasks/  
        recent-notes.md  
    acceptance/  
      recent-notes.md
```



```
traces/
  recent-notes.md
evaluations/
  recent-notes.md
progress/
  current.md
  history.md
pipelines/
  catalog.md
```

README.md

```
# Agentic Notes
```

CLI mínima para gestionar notas locales y demostrar un flujo de trabajo con IA, SDD, subagentes y harness.

```
## Comandos
```

```
```bash
npm test
npm run validate:sdd
npm run check
node src/cli.mjs add "Investigar AGENTS.md"
node src/cli.mjs recent --limit 5
node src/cli.mjs search AGENTS
```
```

```
## Flujo de trabajo
```

1. El requerimiento vive en ``.github/orquestador/sdd/requirements/``.
2. La spec verificable vive en ``.github/orquestador/sdd/specs/``.
3. Las tareas viven en ``.github/orquestador/sdd/tasks/``.
4. La trazabilidad vive en ``.github/orquestador/sdd/traces/``.
5. El estado operativo vive en ``.github/orquestador/sdd/progress/``.
6. El cierre se valida con ``.init.sh`` o ``npm run check``.

package.json

```
{
  "name": "agentic-notes",
  "version": "1.0.0",
  "type": "module",
  "private": true,
  "scripts": {
    "test": "node --test",
    "validate:sdd": "node scripts/validate-sdd.mjs",
```

```

    "eval": "node harness/run-eval.mjs",
    "check": "npm test && npm run validate:sdd && npm run eval"
  },
  "engines": {
    "node": ">=20"
  }
}

```

AGENTS.md

AGENTS.md

Este repositorio usa IA con una regla central: ningún cambio importante se cierra sin spec, evidencia y trazabilidad.

Lectura inicial

1. Leer `README.md`.
2. Leer `.github/orquestador/context/product.md`.
3. Leer `.github/orquestador/context/architecture.md`.
4. Si la tarea toca comportamiento, leer la spec correspondiente en `.github/orquestador/sdd/specs/`.
5. Revisar `.github/orquestador/sdd/progress/current.md` antes de actuar.

Roles

- `explorer`: investiga y entrega hallazgos con evidencia. No modifica archivos.
- `spec\_author`: convierte una idea en requirements, spec y tasks. No implementa.
- `implementer`: modifica solo los archivos bajo ownership explícito.
- `reviewer`: revisa diff, tests y trace. No corrige mientras revisa.

Reglas de cierre

- Ejecutar `npm run check`.
- Actualizar trace si cambian requirements, tasks o tests.
- Registrar evidencia en `.github/orquestador/sdd/progress/history.md`.
- No declarar `done` si `init.sh` o `npm run check` fallan.

init.sh

```

#!/usr/bin/env sh
set -eu

command -v node >/dev/null 2>&1 || {
  echo "Node.js no está disponible"
  exit 2
}

```

```
npm test || exit 1
npm run validate:sdd || exit 3
npm run eval || exit 1
```

src/store.mjs

```
import fs from "node:fs";
import path from "node:path";

const dataDir = path.resolve(".data");
const dataFile = path.join(dataDir, "notes.json");

export function readNotes() {
  if (!fs.existsSync(dataFile)) return [];
  return JSON.parse(fs.readFileSync(dataFile, "utf8"));
}

export function writeNotes(notes) {
  fs.mkdirSync(dataDir, { recursive: true });
  fs.writeFileSync(dataFile, JSON.stringify(notes, null, 2) + "\n", "utf8");
}

export function addNote(title) {
  if (!title || !title.trim()) {
    throw new Error("El título de la nota es obligatorio");
  }

  const notes = readNotes();
  const now = new Date().toISOString();
  const note = {
    id: `note-${notes.length + 1}`,
    title: title.trim(),
    created_at: now,
    updated_at: now
  };

  notes.push(note);
  writeNotes(notes);
  return note;
}

export function recentNotes(limit = 10) {
  const parsedLimit = Number(limit);
  if (!Number.isInteger(parsedLimit) || parsedLimit < 1 || parsedLimit > 50) {
    throw new Error("El límite debe ser un entero entre 1 y 50");
  }

  return readNotes()

```

```

    .toSorted((a, b) => b.updated_at.localeCompare(a.updated_at))
    .slice(0, parsedLimit);
}

export function searchNotes(query) {
  const normalized = String(query || "").trim().toLowerCase();
  if (!normalized) return [];
  return readNotes().filter((note) =>
    note.title.toLowerCase().includes(normalized)
  );
}

```

src/cli.mjs

```

#!/usr/bin/env node
import { addNote, recentNotes, searchNotes } from "./store.mjs";

const [, , command, ...args] = process.argv;

function getOption(name, fallback) {
  const index = args.indexOf(name);
  if (index === -1) return fallback;
  return args[index + 1] ?? fallback;
}

function printNotes(notes) {
  if (notes.length === 0) {
    console.log("No hay notas para mostrar.");
    return;
  }

  for (const note of notes) {
    console.log(`${note.id} | ${note.updated_at} | ${note.title}`);
  }
}

try {
  if (command === "add") {
    const note = addNote(args.join(" "));
    console.log(`Nota creada: ${note.id}`);
  } else if (command === "recent") {
    printNotes(recentNotes(getOption("--limit", 10)));
  } else if (command === "search") {
    printNotes(searchNotes(args.join(" ")));
  } else {
    console.log("Uso: notes add <titulo> | recent [--limit n] | search <texto>");
    process.exitCode = 2;
  }
} catch (error) {

```

```
console.error(error.message);
process.exitCode = 1;
}
```

test/notes.test.mjs

```
import assert from "node:assert/strict";
import fs from "node:fs";
import test from "node:test";
import { addNote, recentNotes, searchNotes } from "../src/store.mjs";

test.beforeEach(() => {
  fs.rmSync(".data", { recursive: true, force: true });
});

test("recent muestra 10 notas por defecto", () => {
  for (let index = 1; index <= 12; index += 1) {
    addNote(`Nota ${index}`);
  }

  assert.equal(recentNotes().length, 10);
});

test("recent respeta --limit", () => {
  for (let index = 1; index <= 8; index += 1) {
    addNote(`Nota ${index}`);
  }

  assert.equal(recentNotes(5).length, 5);
});

test("recent rechaza límites inválidos", () => {
  assert.throws(() => recentNotes(0), /entero entre 1 y 50/);
  assert.throws(() => recentNotes(99), /entero entre 1 y 50/);
});

test("search encuentra notas por título", () => {
  addNote("Investigar AGENTS.md");
  addNote("Preparar runbook");

  assert.equal(searchNotes("agents").length, 1);
});
```

scripts/validate-sdd.mjs

```
import fs from "node:fs";

const requiredFiles = [
  ".github/orquestador/context/product.md",
  ".github/orquestador/context/architecture.md",
  ".github/orquestador/sdd/requirements/recent-notes.md",
  ".github/orquestador/sdd/specs/recent-notes.md",
  ".github/orquestador/sdd/tasks/recent-notes.md",
  ".github/orquestador/sdd/acceptance/recent-notes.md",
  ".github/orquestador/sdd/traces/recent-notes.md"
];

let failed = false;

for (const file of requiredFiles) {
  if (!fs.existsSync(file)) {
    console.error(`Falta archivo SDD: ${file}`);
    failed = true;
  }
}

const trace = fs.existsSync(".github/orquestador/sdd/traces/recent-notes.md")
  ? fs.readFileSync(".github/orquestador/sdd/traces/recent-notes.md", "utf8")
  : "";

for (const requirement of ["R1", "R2", "R3"]) {
  if (!trace.includes(requirement)) {
    console.error(`Trace incompleta: falta ${requirement}`);
    failed = true;
  }
}

process.exit(failed ? 3 : 0);
```

.github/orquestador/context/product.md

Producto

Agentic Notes es una CLI local para crear y consultar notas. Su objetivo no es competir con una app de notas completa, sino demostrar una base robusta para trabajo asistido por IA.

Usuarios

- Desarrolladores que quieren registrar ideas rápidas.
- Equipos que quieren practicar SDD y trazabilidad.
- Personas que necesitan un ejemplo mínimo pero funcional.

No objetivos

- Sin sincronización cloud.
- Sin UI web.
- Sin autenticación.

[.github/orquestador/context/architecture.md](https://github.com/orquestador/context/architecture.md)

Arquitectura

La aplicación usa Node.js ESM y almacenamiento JSON local.

Componentes

- `src/cli.mjs`: frontera de comandos.
- `src/store.mjs`: reglas de dominio y persistencia.
- `test/notes.test.mjs`: cobertura de comportamiento.
- `scripts/validate-sdd.mjs`: gate local de estructura SDD.
- `harness/run-eval.mjs`: evaluación mínima por fixtures.

Regla

La CLI puede parsear argumentos, pero las reglas de negocio viven en `store.mjs`.

SDD: requirements

Requirements: recent notes

R1

El sistema debe mostrar las 10 notas más recientes por defecto.

R2

El usuario debe poder cambiar el límite con `--limit`, entre 1 y 50.

R3

Si no hay notas, el sistema debe devolver un mensaje neutro sin fallar.

SDD: spec

```
# Spec: recent notes

Comando:

```bash
node src/cli.mjs recent --limit 5
```

## Reglas

- Ordenar por `updated_at` descendente.
- Usar límite 10 si no se informa `--limit`.
- Rechazar límites menores a 1 o mayores a 50.
- Mostrar `No hay notas para mostrar.` cuando no existan notas.
```

SDD: tasks

```
# Tasks: recent notes

- [x] T1 -> R1: implementar orden descendente y límite por defecto.
- [x] T2 -> R2: parsear y validar `--limit`.
- [x] T3 -> R3: cubrir estado sin notas.
- [x] T4 -> R1/R2/R3: agregar tests.
- [x] T5 -> R1/R2/R3: actualizar trace.
```

SDD: acceptance

```
# Acceptance: recent notes

- A1: con 12 notas, `recent` muestra 10.
- A2: con `--limit 5`, `recent` muestra 5.
- A3: con `--limit 0`, el comando falla con mensaje claro.
- A4: sin notas, el comando muestra un mensaje neutro.
```

SDD: traces

```
# Trace: recent notes

Requirement	Task	Test	Evidencia
R1	T1, T4	`recent muestra 10 notas por defecto`	`npm test`
```



```
R2	T2, T4	`recent respeta --limit`	`npm test`
R2	T2, T4	`recent rechaza límites inválidos`	`npm test`
R3	T3, T4	salida vacía de `printNotes`	`npm test` + revisión CLI
```

Harness

`harness/fixtures/recent.jsonl`

```
{ "id": "recent-default", "command": "recent", "expected": "10 notas por defecto" }  
{ "id": "recent-limit", "command": "recent --limit 5", "expected": "5 notas" }  
{ "id": "recent-empty", "command": "recent", "expected": "mensaje neutro si no hay notas" }
```

`harness/evals/rubric.md`

```
# Rúbrica  
  
- Fidelidad: cumple R1, R2 y R3.  
- Seguridad: no escribe fuera de `.data/`.  
- Claridad: errores con mensaje accionable.  
- Mantenibilidad: reglas en `store.mjs`, CLI en `cli.mjs`.  
  
Resultado mínimo aceptable: ninguna dimensión crítica en rojo.
```

`harness/run-eval.mjs`

```
import fs from "node:fs";  
  
const fixtures = fs  
  .readFileSync("harness/fixtures/recent.jsonl", "utf8")  
  .trim()  
  .split("\n")  
  .map((line) => JSON.parse(line));  
  
for (const fixture of fixtures) {  
  if (!fixture.id || !fixture.command || !fixture.expected) {  
    console.error(`Fixture inválido: ${JSON.stringify(fixture)}`);  
    process.exit(1);  
  }  
}  
  
console.log(`Fixtures válidos: ${fixtures.length}`);
```

Prompts reutilizables

`.github/prompts/plan-feature.prompt.md`

```
---
agent: 'agent'
description: "Convertir una idea en spec SDD"
---

Usá contexto de `.github/orquestador/context/`.

Entrada:
${input:feature:Describe la feature}

Producí:
1. problema;
2. no objetivos;
3. requirements R1..Rn;
4. spec verificable;
5. tasks por slices;
6. tests sugeridos;
7. riesgos.

No implementes.
```

`.github/prompts/review-pr.prompt.md`

```
---
agent: 'agent'
description: "Revisar PR con foco en comportamiento y trazabilidad"
---

Revisá diff, tests y SDD.

Prioridad:
1. bugs;
2. regresiones;
3. permisos;
4. requirements sin test;
5. tests sin requirement.

Salida:
- findings con archivo;
- preguntas abiertas;
- decisión: aprobar, bloquear o pedir cambios.
```

GitHub Actions

`.github/workflows/ci.yml`

```
name: ci
```

```
on:
  pull_request:
  push:
    branches: [main]

permissions:
  contents: read

jobs:
  check:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 20
      - run: npm run check
```

Progress

[.github/orquestador/sdd/progress/current.md](#)

```
# Current

Feature activa: recent notes
Estado: implementado
Bloqueos: ninguno
Siguiendo paso: revisión humana y cierre.
```

[.github/orquestador/sdd/progress/history.md](#)

```
# History

## recent notes

- Requirements R1, R2 y R3 cubiertos.
- Tests ejecutados con `npm test`.
- SDD validado con `npm run validate:sdd`.
- Harness validado con `npm run eval`.
```

Cierre de La Biblia Moderna

La Biblia Moderna no propone usar más IA por usar más IA. Propone trabajar mejor: con intención clara, contexto útil, especificaciones verificables, subagentes bien recortados, prompts reutilizables, harnesses medibles y evidencia de cierre.

Si una persona puede abrir un repositorio, leer sus reglas, entender el contrato, ejecutar la validación y continuar el trabajo sin depender de una conversación perdida, el sistema ya empezó a madurar.

Ese es el objetivo final: que la inteligencia artificial no sea una improvisación brillante, sino una capacidad de ingeniería.